

GammaLoop

Differential cross-sections with Local Unitarity

1 Overview

GammaLoop computes differential collider cross-sections with the Local Unitarity method. It is an application and research codebase rather than a thin numerical library: process generation, model import, integrand construction, integration, observables, persistence, and diagnostics meet in one stateful workflow.

Primary interface

The command-line interface is the primary user interface. A run card creates a state, executes commands, and leaves a reusable working directory. The Rust and Python APIs load the same persisted state and expose selected structured operations; they do not define a second, independent execution model.

1.1 Choose a task

- To create and resume a known state, follow the [first-state tutorial](#).
- To adapt a process specification or generation filters, use the [process-generation manual](#) with the [generated generate reference](#).
- To automate a loaded state or diagnose a long run, choose the [Rust/Python interface guide](#) or [logging and diagnostics guide](#).

1.2 A stateful run

A typical source checkout is built and exercised from the repository root:

```
just build-cli-release
./gammaLoop ./examples/cli/gg_hhh/1L/gg_hhh_1L.toml
```

The run card imports a model, generates the requested process, executes its command blocks, and records the resulting state. Resume that directory explicitly for subsequent work:

```
./gammaLoop -s ./examples/cli/gg_hhh/1L/state \
run integrate_physical -c "quit -o"
```

The persisted run.toml records how to replay the run. The settings files describe global and default runtime configuration, while processes/ contains process-specific state. Ordinary runs create gammaLoop_state/ unless a different state path is supplied.

1.3 Installation and external tools

GammaLoop is currently developed primarily from source. The supported development path is the repository's Nix shell, or a local Rust toolchain together with just, a recent GNU toolchain, and Python 3.11 or newer when building bindings. FORM 4.2.1 or newer is needed for analytical integration of integrated UV counterterms. UFO model import also needs the Python ufo-model-loader package.

```
nix develop
just build-cli
just build-api
```

Diagram rendering is a separate concern and uses Clinnet and Typst. Building the CLI does not imply that these drawing tools are installed.

1.4 Related crates

How the crates fit together

Linnet provides the half-edge graph model and graph algorithms. Spenso provides typed tensors, tensor structures, and network execution. Idenso handles symbolic tensor identities and algebraic simplification. Vakint matches and evaluates vacuum-integral topologies. GammaLoop combines these components into a collider-calculation workflow.

For a deeper view of how the command layer, state, integrands, and external components fit together, see the [architecture overview](#).

1.5 Where to begin

- Use built-in `--help` for the CLI options supported by your installed version.
- Start with the included `gg_hhh` run card for a complete state lifecycle.
- Use the Rust or Python API only when a program needs structured access to loaded state or sample-evaluation results.
- Check prerequisites before starting an expensive integration; some workflows require a Symbolica license or external tools such as FORM.

2 Tutorial

This tutorial builds the GammaLoop command-line interface and uses the included one-loop `g g -> h h h` run card to create a state that can be inspected and resumed. Compilation and process generation can take longer than a `--help` smoke test.

2.1 Prerequisites

Work from a GammaLoop source checkout. The supported development environment is the repository Nix shell; without Nix you need Rust 1.85 or newer, just, a recent GNU toolchain, and the dependencies described in the root README. The bundled Standard Model used below does not need the UFO loader. A Symbolica license may be required by your build and use case.

```
nix develop
just build-cli
./gammaLoop --help
```

The last command should print the one-shot CLI options and command tree. The `./gammaLoop` wrapper selects the binary built under `target/dev-optim/`.

Verification scope and cost

The docs harness checks these shell blocks for syntax; it does not build or execute GammaLoop. Verify a clean checkout manually in `nix develop`. A cold CLI build and process generation are the high-resource path and can take tens of minutes, while reopening a valid state is normally much cheaper. The invariant for this tutorial is a resumable state with the files listed below, not a completed Monte Carlo integration.

2.2 Generate an example process

The run card declares a state folder, settings, and named command blocks. Run its generate block and persist the result on exit:

```
./gammaLoop --clean-state \  
./examples/cli/gg_hhh/1L/gg_hhh_1L.toml \  
run generate -c "quit -o"
```

`--clean-state` removes the resolved state first

Use it only for this first, reproducible run. Omit it when resuming work you want to keep. Its generated argument reference records the exact valueless-flag semantics. The example card resolves its state to `examples/cli/gg_hhh/1L/state` and requests ten worker cores; make a private copy of the card before changing either value.

The block imports `sm-default.json`, generates the selected one-loop pentagon contribution, builds its integrand, saves DOT data, and writes the state. A successful run exits without an error and leaves at least `run.toml`, `global_settings.toml`, `default_runtime_settings.toml`, and `processes/` in the state folder. `run.toml` records the commands and settings needed to replay the run; it is not merely a log file.

2.3 Resume the saved state

Load the saved state and ask the active session to display its processes:

```
./gammaLoop -s ./examples/cli/gg_hhh/1L/state \  
run -c "display processes; quit -o"
```

Success means the display includes the generated `gg_hhh` process and the command exits while keeping the same state. You can now run the card's `integrate_euclidean` or `integrate_physical` block, but those are deliberately beyond the first-success path because they request a substantial Monte Carlo integration.

2.4 Troubleshooting and next steps

- If the wrapper cannot find a binary, rerun just `build-cli` and confirm that `target/dev-optim/gammaLoop` exists.
- If compilation or process generation exhausts local resources, copy the card and reduce the values under `cli_settings.global.n_cores`; do not edit a state midway through generation.
- If startup reports a state fingerprint or settings mismatch, replay the card with a new state folder instead of transplanting only its `processes/` directory.
- Use `./gammaLoop help generate` to inspect the flags supported by your installed version, then continue with the process-generation manual and the `filtered generate-command` reference.

Example run card

See the complete `gg_hhh` run card before adapting the process, state location, or resource settings for your own calculation.

3 Process generation and state workflows

GammaLoop describes a calculation with a TOML run card and stores its reusable state in a directory. Start the command-line interface through the repository wrapper (`./gammaLoop`) or the Cargo-built executable, then generate processes, integrands, and integration jobs within that state.

3.1 Choose a generation mode

The interactive command tree separates cross sections, amplitudes, and operations on processes already stored in the active state:

```
generate xs INITIAL > FINAL ...
generate amp INITIAL > FINAL ...
generate existing ...
generate help xs
```

xs constructs forward-scattering graphs and filters compatible Cutkosky cuts. amp constructs ordinary amplitudes. Their final-state lists therefore have different meanings even when the printed particle syntax is identical. Use `generate help <mode>` inside the stateful CLI to see the flags supported by your installed version.

3.2 Process specification grammar

Process specifications accept `to` or `>` between space-separated initial and final states. Braces express alternatives in a cross-section final state, while `{}` denotes an empty state:

```
e+ e- > mu+ mu-
e+ e- > { Z Z, a a, H H }
{} > {}
```

A slash vetoes particles and a vertical bar selects an allow-list. The words `veto` and `only` are equivalent forms. Amplitude coupling constraints use the coupling name directly; powered constraints apply to the complete cross section:

```
e+ e- > d d~ g / u c QED==2 QCD>=2 QCD<=4
e+ e- > mu+ mu- | g ghG QED^2==2 QCD^2>=2
```

The bracketed perturbative block is intentionally distinct from ordinary flags. `{n}` fixes the amplitude loop count (or the sum across the two cut sides), `{{n}}` fixes the forward-graph loop count, and `QCD=n` or the shorthand `QCD` selects a relative perturbative order:

```
e+ e- > Z [ {1} {{2}} QCD=2 QED=1 ]
e+ e- > Z [ QCD ]
```

Resolve names through the active model

Particle names, antiparticles, coupling-order names, and allowed interactions come from the model loaded in the current state. A syntactically valid specification can still be rejected when that model does not define a requested name.

3.3 Filters, grouping, and multiplicity

Generation flags control topology filters, loop ranges, cut blobs and spectators, symmetrization, and numerator-aware grouping. These choices are physics-significant. In particular, grouping can combine graph multiplicities and numerator ratios; external-fermion symmetrization requires the caller to retain the associated ordering signs. Run `generate help <mode>` before choosing filters to see the available aliases, defaults, values, and argument counts.

The generated graph's overall factor keeps separate contributions for automorphisms, internal fermion loops, external-fermion ordering, numerator-independent symmetry grouping, and numerator-dependent grouping. Persist the state and inspect that factor when auditing diagram normalization instead of replacing it with an assumed factorial.

3.4 A maintainable generation card

The following reduced card uses the repository's scalar model and the same one-loop bubble command exercised by the maintained scalar-topology example. Save it as `manual-bubble.toml` and run `./gammaloop --clean-state manual-bubble.toml run generate -c "quit -o"` from the repository root.

```
commands = []

[cli_settings.state]
folder = "./gammaloop_state/manual-bubble"
name = "manual_bubble"

[[command_blocks]]
name = "generate"
commands = [
    "import model scalars-default.json",
    "generate amp scalar_1 > scalar_1 [{1}] --allowed-vertex-interactions V_3_SCALAR_122 -p bubble -i scalar_bubble",
    "generate",
    "save state -o",
]
```

The acceptance invariant is a persisted state in which `display` processes identifies process bubble and integrand scalar_bubble; replaying the card with a clean destination must produce the same named objects. The exact command and setting semantics are linked from the amplitude-generation reference, state-persistence reference, and state-folder setting. The full maintained source is the scalar bubble run card.

Interpret the first failing stage

An unknown particle or `V_3_SCALAR_122` error means the scalar model was not imported or no longer exposes that interaction. A process with no retained diagram points to the process specification or generation filters. A generated process that disappears after restart means `save state -o` did not complete or the resumed `-s` path differs from `cli_settings.state.folder`.

3.5 From generation to evaluation

A reproducible run keeps its stages explicit:

- import or select the model before resolving a process specification;
- generate the process and inspect the retained diagrams;
- generate the requested integrand representation;
- set kinematics, sampling, stability, and subtraction settings;
- evaluate samples or run an integration command block;
- exit with persistence enabled when the state should be resumed.

Keep the complete state together

`run.toml` records boot settings, reusable command blocks, and top-level commands. Generated process and integrand artifacts live beside it. Resume with `-s STATE_FOLDER`; replay the run card to reconstruct the state from scratch. Copy or archive the entire state directory rather than moving a generated subdirectory on its own, because settings and fingerprints are checked together.

4 Logging and diagnostics

This guide explains the default tracing configuration used by the CLI and API at startup and gives copyable filters for narrowing a noisy calculation to the subsystem and work unit that matter.

Before you start

Prerequisites: a built GammaLoop CLI and a state or run card that reaches the behavior you want to inspect. Filter changes take effect immediately and add negligible setup time, but verbose inspect or dump events can produce large files and slow a run. The expected invariant is that a narrow directive changes diagnostic visibility, not numerical results. If a filter produces nothing, first use a broader target at debug, confirm that release compilation has not removed the callsite, and then add tag constraints. After isolating the event, record the exact directive with the run card and continue to the CLI/settings reference for persistence and state controls.

4.1 Default directives

Normal session defaults come from GlobalSettings:

- `display_directive = "info"`
- `logfile_directive = "off"`

These are the application defaults written into `global_settings.toml` when defaults are shown or persisted.

In practice, `info` is the current convention for normal screen-facing output. Selective internal diagnostics should generally stay on debug and be turned on via narrower target/tag filters rather than by raising the whole display logger.

4.2 Startup precedence

Startup applies logging in this order:

1. Base settings come from GlobalSettings.
2. Environment overrides replace the base spec:
 - `GL_ALL_LOG_FILTER` overrides both stderr and logfile filters.
 - `GL_DISPLAY_FILTER` overrides only the stderr/display filter.
 - `GL_LOGFILE_FILTER` overrides only the logfile filter.
3. CLI overrides are then applied:
 - `-l/--level` overrides the stderr/display filter for the session.
 - `-L/--logfile-level` overrides the logfile filter for the session.
4. Some CLI modes hard-disable logfile logging for the session:
 - `--read-only-state`
 - `--logfile-level off`

When logfile logging is hard-disabled at boot, later settings changes cannot re-enable it for that session.

4.3 CLI level mapping

`-l/--level` and `-L/--logfile-level` expand to explicit crate directives:

- `off` -> `gammaloop_api=off,gammalooprs=off`
- `error` -> `gammaloop_api=error,gammalooprs=error`
- `warn` -> `gammaloop_api=warn,gammalooprs=warn`
- `info` -> `gammaloop_api=info,gammalooprs=info`
- `debug` -> `gammaloop_api=debug,gammalooprs=debug`
- `trace` -> `gammaloop_api=trace,gammalooprs=trace`

4.4 Empty-spec fallback floors

The filter builders also have hardcoded fallback floors used when a spec is empty:

- display/stderr builder fallback: `off`
- logfile builder fallback: `warn`

This is separate from the normal application defaults above. In other words:

- if startup uses the normal settings path, the defaults are `info` for display and `off` for logfile
- if an empty filter string is explicitly parsed, the display builder falls back to `off` and the logfile builder falls back to `warn`

4.5 Runtime updates

Changing `global.display_directive` or `global.logfile_directive` after startup reloads the active tracing filters. The CLI `stderr` override from `-l/--level` is treated as a session override and is separate from the persisted global setting.

4.6 Tag-based debug filtering

For selective debug logging, GammaLoop uses its own logging DSL rather than raw `EnvFilter`.

This follows the general idea in Tom Mrazik's [tag-based logging note](#).

The main reason for owning the DSL is that GammaLoop needs composable tag groups and stable semantics around presence, absence, and explicit boolean values.

- Use stable targets for broad subsystems such as `gammaloops::uv::forest` or `gammaloops::integrands::process::cross_section`.
- Add boolean event fields as composable tags.
- Filter by field presence with `target[{tag_a,tag_b,!tag_c}]=debug`, or by displayed boolean values with `target[{-#tag_a,#!tag_c}]=debug`.

This is preferable to span-name filtering because span-based filters can admit child-library events emitted while the span is active. GammaLoop tag filters only look at the event target and the boolean tag fields on the event itself.

4.6.1 Supported directive syntax

The supported directive forms are:

- `level`
- `target=level`
- `target[{tag_a,tag_b,!tag_c}]=level`
- `target[{-#tag_a,#!tag_c}]=level`
- `target`

Notes:

- Directives are comma-separated.
- `target` is matched as a module-style prefix, so `gammaloops::uv=debug` matches `gammaloops::uv` and `gammaloops::uv::forest`.
- A directive without an explicit `=level` is treated as `=trace`.
- Tag groups are conjunctions: `[{generation,uv}]` means both tags must match.
- `!tag` means the field is absent.
- `#tag` means the field is present with boolean value `true`.
- `#!tag` means the field is present with boolean value `false`.
- `field=value` matches an explicit field value.

4.6.2 Tag matching model

Tags are represented by event fields.

- `#generation` at the callsite emits `generation = true`.
- In a directive, `generation` means the event must carry a field named `generation`.
- In a directive, `#generation` means the event must carry `generation = true`.
- In a directive, `!inspect` means the event must not carry a field named `inspect`.
- In a directive, `#!inspect` means the event must carry `inspect = false`.

- In a directive, `inspect=true`, `mode=summary`, or `label="soft region"` means the event must carry that value specifically.

This means the callsite controls both:

- whether a log is selectable by a presence/absence query
- whether a log can be selected by an explicit value query such as `inspect=false` or `mode=summary`

4.6.3 Static vs dynamic filtering

Presence/absence-only directives stay on the lazy metadata path.

- `generation`
- `!inspect`
- `gammaloops::uv::forest[{generation,uv,!dump}]=debug`

These can be decided from the callsite target and declared field names alone, so the filter can answer always or never at callsite registration time.

Value-matching directives are dynamic.

- `inspect=false`
- `#!inspect`
- `#generation`
- `mode=summary`
- `label="soft region"`

Those depend on the event instance, so they are evaluated in the per-event pass.

4.6.4 Precedence

When multiple directives match the same event, GammaLoop prefers the most specific one:

- longer target prefix wins
- then the directive with more tag requirements wins
- then later directives win if the earlier specificity is identical

This lets broad directives such as `gammaloops=info` coexist with narrow tag-specific directives such as `gammaloops::uv::forest[{generation,uv,dump}]=debug`.

4.6.5 Copy-pasteable display tags

Display logs render boolean fields next to the source as a directive tag group:

```
@gammaloops::uv::forest[{#generation, #uv, #!inspect}] DEBUG: message
```

Every boolean field is rendered this way, not only the standard tag vocabulary. `true` becomes `#field_name`; `false` becomes `#!field_name`. These boolean fields are removed from the normal display field table.

Use the full logging prefix when you want the rendered source to be a valid directive head:

```
gammaloop --logging-prefix full -l debug ...
```

Then the text after `@` and before the log level can be copied into a directive by adding `=debug`, for example:

```
display_directive = "gammaloops::uv::forest[{#generation, #uv, #!inspect}]=debug"
```

This copy-paste form assumes the display source is the module target. Enabling `full_line_source` adds file and line information to the source display, which is useful for locating code but is not a valid directive target.

4.6.6 Sink-only field prefixes

This repo also has sink-routing prefixes for fields:

- `file.<name>`: only rendered into the logfile/json sink
- `display.<name>`: only rendered into the stderr/display sink

Examples:

- `file.integrands = %...` will appear only in the file logger output
- `display.progress = %...` will appear only in the display logger output

These prefixes are a formatting/routing feature, not a separate event kind. They decide where a field is rendered after the event has been emitted. This lets a callsite attach large payloads such as `file.integrands` without dumping them to stderr.

4.6.7 Pipeline-wide tag set

Prefer a small stable vocabulary that cuts across the whole pipeline.

Primary phase tags:

- `generation`: work that builds or prepares runtime objects before Monte Carlo integration starts
- `integration`: work performed while evaluating samples or managing the adaptive integration loop
- `profile`: UV/IR/profile-style diagnostic scans and their analysis
- `persistence`: reading or writing saved state, manifests, checkpoints, and exported results

Core domain tags:

- `uv`: ultraviolet counterterms, UV forests, UV profiles, or UV-specific generation/evaluation logic
- `ir`: infrared subtraction, IR profiles, or IR-specific generation/evaluation logic
- `subtraction`: threshold, UV, or IR subtraction logic when the main concern is subtraction rather than the specific regime
- `sampling`: channel choice, discrete axes, parameterizations, or sample-generation choices
- `stability`: precision escalation, retries, instability diagnosis, and numerical safety checks
- `observables`: histogramming, event-to-observable projection, and observable snapshot production
- `selectors`: event-selection logic and selector decisions
- `cache`: cache lookup, reuse, invalidation, or cache-debug instrumentation

Common work-unit tags:

- `graph`: the log is about one graph or graph-local data
- `group`: the log is about a graph group or another explicitly grouped aggregate
- `orientation`: the log is about orientation-dependent data or choosing/summing orientations
- `channel`: the log is about multi-channeling or a particular channel
- `cut`: the log is about a cut, raised cut, or cut-local computation
- `event`: the log is about generated/retained event objects or event-group processing
- `sample`: the log is about one evaluation sample, its coordinates, or per-sample intermediate values
- `iteration`: the log is about one adaptive integration iteration or iteration-level summaries
- `term`: the log is about one algebraic term, summand, or term-local contribution

Common purpose tags:

- `solver`: the log is about root-finding, linear solves, fitting, or similar numerical solver state
- `compile`: the log is about evaluator/code generation or compilation-oriented preparation
- `inspect`: the log exposes detailed intermediate state for debugging, rather than a high-level milestone
- `summary`: the log is a compact roll-up rather than a step-by-step trace
- `dump`: the log emits large or structured payloads such as expressions, tables, or serialized views

4.6.8 Tag boundaries

Use the most general tag that accurately captures the reason you want to turn the log on or off.

- Prefer generation over compile when the message is a broad generation milestone.
- Add compile only when the message is specifically about evaluator construction or compilation-like work.
- Prefer uv or ir when the regime matters to the user; use subtraction when the subtraction mechanism is the real concern.
- Prefer inspect for verbose intermediate values that are mainly useful while debugging internals.
- Add dump when the payload is large enough that users may want to suppress it separately from lighter debug logs.
- Do not use graph, cut, or sample as substitutes for generation or integration; they refine phase tags rather than replace them.

4.6.9 Naming guidance

- Use phase tags first. They answer “when in the pipeline did this happen?”
- Use domain tags second. They answer “what subsystem is this about?”
- Use work-unit tags third. They answer “what object is being worked on?”
- Use purpose tags last for optional refinement.
- Prefer tags that are meaningful across multiple modules.
- Avoid tags that merely restate a function name.
- Avoid tags tied to one internal representation unless that representation is a stable concept in user-facing debugging.

parametric is usually not a core pipeline tag. It is acceptable as a local refinement when needed, but should not be treated as part of the primary vocabulary unless it becomes a consistently useful cross-cutting concept.

4.6.10 Example queries

- all generation-time UV forest logs:
 - `gammaloops::uv::forest[{generation,uv}]=debug`
- only the large UV forest dumps for orientation-dependent generation logs:
 - `gammaloops::uv::forest[{generation,uv,orientation,dump}]=debug`
- UV forest term-by-term generation logs:
 - `gammaloops::uv::forest[{generation,uv,term}]=debug`
- all integration-time subtraction debug in amplitude evaluation:
 - `gammaloops::integrands::process::amplitude[{integration,subtraction}]=debug`
- per-sample integration inspection in cross-section evaluation:
 - `gammaloops::integrands::process::cross_section[{integration,sample,inspect}]=debug`
- all integration-time cut solver debug in cross-section evaluation:
 - `gammaloops::integrands::process::cross_section[{integration,cut,solver}]=debug`
- integration-time debug excluding inspect-tagged logs:
 - `gammaloops::integrands::process::cross_section[{integration,!inspect}]=debug`
- only events that explicitly set inspect=true:
 - `gammaloops::integrands::process::cross_section[{integration,inspect=true}]=debug`
- only events with a specific textual mode field:
 - `gammaloops::integrands::process::cross_section[{integration,mode=summary}]=debug`

Example display directive:

```
[cli_settings.global]
display_directive =
"gamma_loop_api=info,gammaloops=info,symbolica=off,poly::gcd=off,gammaloops::uv::forest[{generation,uv,orientation,dump}]=debug"
```

4.7 Unsupported EnvFilter syntax

GammaLoop no longer treats the directive string as generic EnvFilter syntax.

In particular, do not rely on:

- span-name filters such as `target[span_name]=debug`
- generic field filters such as `[{field=value}]`
- `EnvFilter` regex semantics

If those are needed later, they must be added explicitly to the GammaLoop DSL.

4.8 Release-build note

`gammaloops` is compiled with tracing feature `release_max_level_info`, so `debug!` and `trace!` callsites in that crate are compiled out in release builds.

4.9 Authoritative implementation

The startup precedence and sink wiring live in the [API tracing setup](#). The shared directive parser, CLI level mapping, and filter behavior live in the [runtime tracing utilities](#), while persisted defaults are owned by `GlobalSettings`.

5 Kurvst curve and path API

Kurvst is GammaLoop's Typst-facing WebAssembly layer for Bezier and Hobby paths, arc-length trimming, sampled path patterns, parallel curves, and CeTZ conversion. This page embeds the complete maintained Kurvst manual used by the drawing pipeline.

Geometry rather than topology

Kurvst transforms curve geometry. Linnet and Linnest own graph topology and layout; GammaLoop's drawing templates combine those results with Kurvst paths and styles.

`kurvst` is a Typst package backed by `kurvst.wasm`, a small Kurbo-based plugin. Its public API is path-based: construct a path dictionary, pass that path into transforms, and draw the returned path.

It exposes Bezier utilities that are awkward or unavailable in native Typst:

- constructing cubic and Hobby paths,
- trimming paths by arc length,
- generating sampled path patterns,
- fitting Kurbo offset/parallel paths,
- converting returned path geometry to CeTZ drawing commands.

Path geometry points are two-item tuples:

```
#let p = (1.2, -0.4)
```

Cubic segments use `start`, `control-start`, `control-end`, and `end`:

```
#let segment = (
  start: (0, 0),
  control-start: (1, 0.5),
  control-end: (2, -0.5),
  end: (3, 0),
)
```

Use `from-cubic(segment)` once to turn that cubic dictionary into a path dictionary. Returned paths contain a curve-command wire path in `path`. Rust converts that wire path to Kurbo's `BezPath` internally, and every path-level Kurvst function accepts either a returned dictionary or the raw wire path.

```
#let path = kurvst.from-cubic(segment)
#let trimmed = kurvst.trim(path, start-outset: 0.2, end-outset: 0.1)
#let shifted = kurvst.parallel(trimmed, distance: 0.15)
```

5.1 Path Wire Format

A Kurvst wire path is a dictionary with one elements array. The elements mirror Typst's native curve commands, but points are plain numeric tuples:

```
#let path = (
  elements: (
    (kind: "move", start: (0, 0)),
    (kind: "line", end: (0.6, 0.3)),
    (kind: "quad", control: (1.0, 1.0), end: (1.4, 0.3)),
    (
      kind: "cubic",
      control-start: (1.8, -0.4),
      control-end: (2.4, 1.0),
      end: (3.0, 0),
    ),
    (kind: "close", mode: "straight"),
  ),
)
```

The supported element kinds are:

- move: starts a subpath at start.
- line: adds a straight segment ending at end.
- quad: adds a quadratic segment using control and end.
- cubic: adds a cubic segment using control-start, control-end, and end.
- close: closes the current subpath. mode is optional and defaults to "straight" when emitted by Kurvst.

Prefer the path fragment helpers for new code. line, quad, and cubic include their start points, so fragments can be built independently:

```
#let custom = kurvst.path(
  kurvst.line((0, 0), (0.6, 0.3)),
  kurvst.quad((0.6, 0.3), (1.0, 1.0), (1.4, 0.3)),
  kurvst.cubic((1.4, 0.3), (1.8, -0.4), (2.4, 1.0), (3.0, 0)),
)
```

path and append flatten fragments. If an appended fragment starts where the current path ends, its leading move is skipped; if it starts elsewhere, the move begins a new subpath. The edited path can go straight back into Kurvst transforms:

```
#let base = kurvst.from-cubic(segment)
#let extended = kurvst.append(base, kurvst.line(segment.end, (3.4, 0.2)))

#let trimmed = kurvst.trim(extended, start-outset: 0.15)
#let shifted = kurvst.parallel(trimmed, distance: 0.1)

#kurvst.to-native(shifted, unit: 36pt, stroke: rgb("#1b7f4c") + 0.7pt)
```

Use elements when you need the raw command stream, points for the visited endpoints, segments for cubic segment dictionaries, to-native for native Typst drawing, and to-cetz-data or to-cetz for CeTZ.

A typical chain keeps the returned dictionaries intact until the final drawing step:

```
#let base = kurvst.hobby-through(demo-start, demo-through, demo-end)
#let trimmed = kurvst.trim(base, start-outset: 0.2, end-outset: 0.1)
#let shifted = kurvst.parallel(trimmed, distance: 0.15)

#cetz.canvas({
  kurvst.to-cetz(base, stroke: rgb("#bbbbbb") + 0.4pt)
  kurvst.to-cetz(trimmed, stroke: rgb("#d72638") + 0.6pt)
  kurvst.to-cetz(shifted, stroke: rgb("#1b7f4c") + 0.6pt)
})
```

5.2 Paths And Trimming

cubic constructs a path from one cubic Bezier. trim trims by curve distance from the start and/or end and returns another path dictionary.

```
#let path = kurvst.from-cubic(segment)
#let trimmed = kurvst.trim(path, start-outset: 0.15, end-outset: 0.1)
```

hobby-through(start, through, end) builds a smooth open curve through three points; segments returns the two cubic halves split at through. hobby-spline(points) does the same for any open point sequence with at least two points. Both return path dictionaries and can be passed directly to path-level helpers.

```
#let spline = kurvst.hobby-spline((
  (0.0, 0.0),
  (0.9, 0.8),
  (1.8, -0.3),
  (2.8, 0.4),
))
#let shifted = kurvst.parallel(spline, distance: 0.14)
#let wiggle = kurvst.pattern(
  spline,
  pattern: kurvst.wave(samples-per-period: 24),
  amplitude: 0.1,
  wavelength: 0.55,
)
#cetz.canvas({
  kurvst.to-cetz(spline, stroke: rgb("#bbbbbb") + 0.45pt)
  kurvst.to-cetz(shifted, stroke: rgb("#1b7f4c") + 0.6pt)
  kurvst.to-cetz(wiggle, stroke: rgb("#355c9a") + 0.55pt)
})
```

split-through combines Hobby spline construction with endpoint trimming and returns one path part between each consecutive point. Consumers such as Linnest can wrap it for graph-edge geometry.

5.3 Path Patterns

pattern generates a one-dimensional pattern along any path dictionary. Built-ins are regular Typst dictionaries:

```
#let wave = kurvst.wave()
#let zigzag = kurvst.zigzag()
#let coil = kurvst.coil(longitudinal-scale: 1.6)
```

String names "wave", "zigzag", and "coil" are accepted for convenience, but they are resolved in Typst before calling wasm.

Custom patterns use the same object shape:

```
#let hook = (
  kind: "points",
  name: "hook",
  interpolation: "linear",
  points: (
    (at: 0, x: 0, y: 0),
    (at: 0.25, x: 0.15, y: 1),
    (at: 0.5, x: 0, y: 0),
    (at: 0.75, x: -0.15, y: -1),
    (at: 1, x: 0, y: 0),
  ),
)
```

`at` is the phase position inside one wavelength, from 0 to 1. The `x` coordinate offsets along the path tangent and `y` offsets along the path normal; both are scaled by amplitude. If `at` is omitted, points are spaced evenly. Use `interpolation: "linear"` for corners and `"smooth"` for a spline through sampled points.

`endpoint-ramp: true` tapers amplitude at anchored endpoints. This is useful for coils, whose longitudinal offset would otherwise put the first and last visible points inside the turn near nodes.

```
#let base = kurvst.from-cubic(segment)
#let path = kurvst.pattern(
  base,
  pattern: hook,
  amplitude: 0.15,
  wavelength: 0.7,
)
#native-scene({
  native-cubics(kurvst.segments(base), stroke: rgb("#c8c8c8") + 0.45pt)
  native-polyline(kurvst.points(path), stroke: rgb("#1b7f4c") + 0.8pt)
})
```

5.4 Parallel Paths

`parallel` uses Kurbo's offset curve fitter to produce a path at a fixed normal distance from the source path. Positive distances follow the left normal of the path direction; negative distances follow the right normal. `start-outset` and `end-outset` trim the fitted path by arc length after the offset is computed.

```
#let base = kurvst.from-cubic(segment)
#let left = kurvst.parallel(base, distance: 0.18, start-outset: 0.35, end-outset: 0.35)
#let right = kurvst.parallel(base, distance: -0.18)
#native-scene({
  native-cubics(kurvst.segments(base), stroke: rgb("#c8c8c8") + 0.45pt)
  native-cubics(kurvst.segments(left), stroke: rgb("#1b7f4c") + 0.8pt)
  native-cubics(kurvst.segments(right), stroke: rgb("#355c9a") + 0.8pt)
})
```

5.5 Path Layers

`layer` is a convenience wrapper for drawing derived visible paths. It combines side-aware offsetting, endpoint outsets, and centered shortening into one path-in/path-out operation. `length` is a fixed visible arc length, `ratio` is a fraction of the input path length, and `resolve-length` decides how to combine them. The default `"min"` keeps whichever limit is shorter.

```
#let base = kurvst.hobby-spline((
  (0.0, 0.0),
```

```

    (0.9, 0.8),
    (1.8, -0.3),
    (2.8, 0.4),
  ))
  #let layer = kurvst.layer(
    base,
    offset: 0.16,
    length: 1.6,
    ratio: 0.5,
    resolve-length: "min",
  )
  #cetz.canvas({
    kurvst.to-cetz(base, stroke: rgb("#bbbbbb") + 0.45pt)
    kurvst.to-cetz(layer, stroke: rgb("#1b7f4c") + 0.8pt)
  })

```

Use side-point to choose the sign of the offset from a point on the desired side of the path. Drawing packages can use this to place derived layers on the same side as an edge label without knowing anything about the physics or graph style that requested the layer.

5.6 Native Drawing Primitives

Kurvst returns dictionaries and arrays. The core geometry can be drawn without CeTZ by emitting native curve content:

```

#kurvst.to-native(kurvst.from-cubic(segment),
unit: 36pt, stroke: black + 0.7pt)

```



The generated reference below keeps drawing code inline. It only uses the small common helpers above for coordinate scaling, native curve construction, point markers, and fixed-size preview blocks.

5.7 CeTZ Interoperability

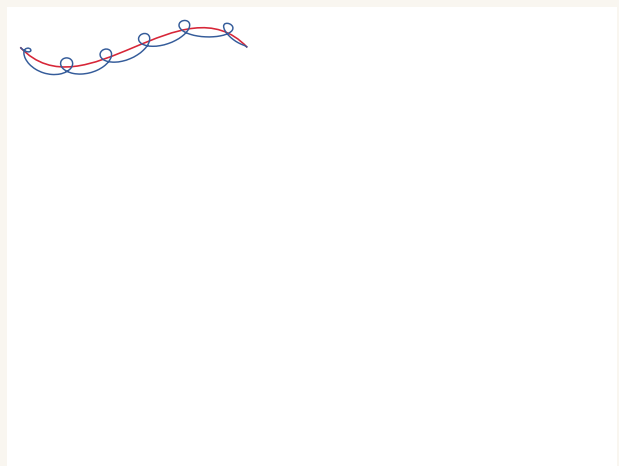
The CeTZ helpers are thin adapters over the same returned geometry. Use them when the surrounding document already lives in a CeTZ canvas, or when you want CeTZ path merging and styling:

```

#cetz.canvas({
  let base = kurvst.from-cubic(segment)
  kurvst.to-cetz(base, stroke: rgb("#d72638")
+ 0.6pt)

  let path = kurvst.pattern(
    base,
    pattern: kurvst.coil(longitudinal-scale:
1.6),
    amplitude: 0.12,
    wavelength: 0.55,
  )
  kurvst.to-cetz(path, stroke: rgb("#355c9a")
+ 0.55pt)
})

```



5.8 Generated Reference

5.9 kurvst

- `append()`
- `center-outset()`
- `close()`
- `coil()`
- `cubic()`
- `cubic-point()`
- `cubic-tangent()`
- `cubic-to()`
- `elements()`
- `from-cubic()`
- `from-elements()`
- `hobby-spline()`
- `hobby-through()`
- `layer()`
- `length()`
- `line()`
- `line-segment()`
- `line-to()`
- `move-to()`
- `outset-point()`
- `parallel()`
- `path()`
- `pattern()`
- `point()`
- `points()`
- `quad()`
- `quad-to()`
- `resolve-length()`
- `segments()`
- `split-through()`
- `to-cetz()`
- `to-cetz-data()`
- `to-native()`
- `trim()`
- `wave()`
- `zigzag()`

Variables

- `layer-defaults`

5.9.1 append

Return a path with additional fragments or elements appended.

5.9.1.1 Parameters

```
append(  
  path: dictionary ,  
  ..parts: any  
) -> dictionary
```


5.9.1.1.1 **path** dictionary

Base Kurvst path dictionary.

5.9.1.1.2 **..parts** any

Path fragments, path elements, or element arrays to append.

5.9.2 **center-outset**

Compute the symmetric trim needed to center a shorter path layer.

5.9.2.1 **Parameters**

```
center-outset(  
  base-length: int float ,  
  length: none int float ,  
  ratio: none int float ,  
  resolve-length: string function ,  
  start-outset: int float ,  
  end-outset: int float  
) -> int float
```

5.9.2.1.1 **base-length** int or float

Full base path arc length.

5.9.2.1.2 **length** none or int or float

Fixed target visible length.

Default: none

5.9.2.1.3 **ratio** none or int or float

Relative target visible length as a fraction of base-length.

Default: none

5.9.2.1.4 **resolve-length** string or function

Resolution strategy for fixed and relative targets.

Default: "min"

5.9.2.1.5 start-outset `int` or `float`

Already-applied trim at the start of the path.

Default: `0`

5.9.2.1.6 end-outset `int` or `float`

Already-applied trim at the end of the path.

Default: `0`

5.9.3 close

Build a close path element.

5.9.3.1 Parameters

```
close(mode: string) -> dictionary
```

5.9.3.1.1 mode `string`

Native Typst curve close mode.

Default: `"straight"`

5.9.4 coil

A smooth coil path pattern.

5.9.4.1 Parameters

```
coil(  
  samples-per-period: int,  
  longitudinal-scale: int float  
) -> dictionary
```

5.9.4.1.1 samples-per-period `int`

Number of samples used to approximate one coil period.

Default: `16`

5.9.4.1.2 longitudinal-scale `int` or `float`

Horizontal scale of the coil before it is mapped onto a path.

Default: `1.25`

5.9.5 cubic

Build a cubic path fragment.

5.9.5.1 Parameters

```
cubic(  
    start: array ,  
    control-start: array ,  
    control-end: array ,  
    end: array  
) -> dictionary
```

5.9.5.1.1 start array

Start point.

5.9.5.1.2 control-start array

Cubic control point near start.

5.9.5.1.3 control-end array

Cubic control point near end.

5.9.5.1.4 end array

Endpoint.

5.9.6 cubic-point

Evaluate a cubic segment at parameter t.

5.9.6.1 Parameters

```
cubic-point(  
    segment: dictionary ,  
    t: int float  
) -> array
```

5.9.6.1.1 segment dictionary

Segment with start, control-start, control-end, and end.

5.9.6.1.2 t int or float

Segment parameter in the range [0, 1].

5.9.7 cubic-tangent

Evaluate the tangent of a cubic segment at parameter t.

5.9.7.1 Parameters

```
cubic-tangent(  
  segment: dictionary ,  
  t: int | float  
) -> array
```

5.9.7.1.1 segment dictionary

Segment with start, control-start, control-end, and end.

5.9.7.1.2 t int or float

Segment parameter in the range [0, 1].

5.9.8 cubic-to

Build a cubic path element.

5.9.8.1 Parameters

```
cubic-to(  
  control-start: array ,  
  control-end: array ,  
  end: array  
) -> dictionary
```

5.9.8.1.1 control-start array

Cubic control point near the start point.

5.9.8.1.2 control-end array

Cubic control point near the endpoint.

5.9.8.1.3 end array

Cubic endpoint.

5.9.9 elements

Return the command elements that make up a Kurvst path.

5.9.9.1 Parameters

```
elements(path: dictionary) -> array
```

5.9.9.1.1 path dictionary

Kurvst path dictionary to inspect.

5.9.10 from-cubic

Build a path fragment from a cubic segment dictionary.

5.9.10.1 Parameters

```
from-cubic(segment: dictionary) -> dictionary
```

5.9.10.1.1 segment dictionary

Segment with start, control-start, control-end, and end.

5.9.11 from-elements

Build a path dictionary from an existing element array.

5.9.11.1 Parameters

```
from-elements(elements: array) -> dictionary
```

5.9.11.1.1 elements array

Array of Kurvst path elements.

5.9.12 hobby-spline

Construct a Hobby spline through an arbitrary point sequence.

5.9.12.1 Parameters

```
hobby-spline(  
  points: array,  
  omega: float,  
  accuracy: float  
) -> dictionary
```

5.9.12.1.1 points array

Two or more points for the open spline to pass through.

5.9.12.1.2 omega float

Hobby curl/tension parameter.

Default: 1.0

5.9.12.1.3 accuracy float

Geometry approximation accuracy passed to the Rust geometry engine.

Default: 0.001

5.9.13 hobby-through

Construct a cubic Hobby path through three points.

5.9.13.1 Parameters

```
hobby-through(  
  start: array ,  
  through: array ,  
  end: array ,  
  omega: float ,  
  accuracy: float  
) -> dictionary
```

5.9.13.1.1 start array

Start point.

5.9.13.1.2 through array

Intermediate point that the curve passes through.

5.9.13.1.3 end array

Endpoint.

5.9.13.1.4 **omega** float

Hobby curl/tension parameter.

Default: 1.0

5.9.13.1.5 **accuracy** float

Geometry approximation accuracy passed to the Rust geometry engine.

Default: 0.001

5.9.14 **layer**

Build a derived visible path layer.

5.9.14.1 **Parameters**

```
layer(  
  path: dictionary ,  
  offset: int float ,  
  length: none int float ,  
  ratio: none int float ,  
  resolve-length: string function ,  
  start-outset: int float ,  
  end-outset: int float ,  
  side-point: none array ,  
  accuracy: float ,  
  optimize: bool  
) -> dictionary
```

5.9.14.1.1 **path** dictionary

Base Kurvst path dictionary.

5.9.14.1.2 **offset** int or float

Signed normal offset distance.

Default: 0

5.9.14.1.3 **length** none or int or float

Fixed target visible length.

Default: none

5.9.14.1.4 ratio `none` or `int` or `float`

Relative target visible length as a fraction of the base length.

Default: `none`

5.9.14.1.5 resolve-length `string` or `function`

Resolution strategy for fixed and relative targets.

Default: `"min"`

5.9.14.1.6 start-outset `int` or `float`

Arc length removed from the start.

Default: `0`

5.9.14.1.7 end-outset `int` or `float`

Arc length removed from the end.

Default: `0`

5.9.14.1.8 side-point `none` or `array`

Optional point used to choose the sign of offset.

Default: `none`

5.9.14.1.9 accuracy `float`

Geometry approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

5.9.14.1.10 optimize `bool`

Let Kurbo simplify/optimize fitted parallel paths.

Default: `true`

5.9.15 length

Compute the arc length of a path dictionary.

5.9.15.1 Parameters

```
length(  
  path: dictionary ,  
  accuracy: float  
) -> int float
```

5.9.15.1.1 path dictionary

Kurvst path dictionary to measure.

5.9.15.1.2 accuracy float

Arc-length approximation accuracy passed to the Rust geometry engine.

Default: 0.001

5.9.16 line

Build a straight-line path fragment.

5.9.16.1 Parameters

```
line(  
  start: array ,  
  end: array  
) -> dictionary
```

5.9.16.1.1 start array

Start point.

5.9.16.1.2 end array

Endpoint.

5.9.17 line-segment

Build a cubic segment dictionary for a straight line.

5.9.17.1 Parameters

```
line-segment(  
  start: array ,  
  end: array  
) -> dictionary
```

5.9.17.1.1 start array

Start point.

5.9.17.1.2 end array

Endpoint.

5.9.18 line-to

Build a line path element.

5.9.18.1 Parameters

```
line-to(end: array) -> dictionary
```

5.9.18.1.1 end array

Line endpoint.

5.9.19 move-to

Build a move path element.

5.9.19.1 Parameters

```
move-to(start: array) -> dictionary
```

5.9.19.1.1 start array

New current point and subpath start.

5.9.20 outset-point

Return from moved toward toward by distance.

5.9.20.1 Parameters

```
outset-point(  
  from: array ,  
  toward: array ,  
  distance: int float  
) -> array
```

5.9.20.1.1 from array

Point to move.

5.9.20.1.2 toward array

Target point that defines the direction.

5.9.20.1.3 distance int or float

Distance to move from from toward toward.

Default: 0

5.9.21 parallel

Generate a parallel path for a path.

5.9.21.1 Parameters

```
parallel(  
  path: dictionary ,  
  distance: int float ,  
  start-outset: int float ,  
  end-outset: int float ,  
  accuracy: float ,  
  optimize: bool  
) -> dictionary
```

5.9.21.1.1 path dictionary

Base Kurvst path dictionary.

5.9.21.1.2 distance int or float

Signed normal offset distance.

Default: 0

5.9.21.1.3 start-outset int or float

Arc length removed from the start before offsetting.

Default: 0

5.9.21.1.4 end-outset `int` or `float`

Arc length removed from the end before offsetting.

Default: `0`

5.9.21.1.5 accuracy `float`

Geometry approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

5.9.21.1.6 optimize `bool`

Let Kurbo simplify/optimize the fitted path.

Default: `true`

5.9.22 path

Build a path dictionary from path fragments or elements.

5.9.22.1 Parameters

```
path(..parts: any) -> dictionary
```

5.9.22.1.1 ..parts `any`

Path fragments, path elements, or element arrays to concatenate.

5.9.23 pattern

Apply a repeated path pattern to a base path.

5.9.23.1 Parameters

```
pattern(  
  path: dictionary,  
  pattern: string dictionary,  
  amplitude: int float,  
  wavelength: int float,  
  phase: int float,  
  samples-per-period: int,  
  coil-longitudinal-scale: int float,  
  anchor-start: bool,  
  anchor-end: bool,  
  accuracy: float  
) -> dictionary
```

5.9.23.1.1 **path** dictionary

Base Kurvst path dictionary.

5.9.23.1.2 **pattern** string or dictionary

Pattern dictionary or built-in pattern name.

Default: "wave"

5.9.23.1.3 **amplitude** int or float

Normal amplitude of the pattern in path units.

Default: 0.1

5.9.23.1.4 **wavelength** int or float

Arc length of one pattern period.

Default: 1.0

5.9.23.1.5 **phase** int or float

Initial phase offset in pattern periods.

Default: 0

5.9.23.1.6 **samples-per-period** int

Samples per period for string-resolved smooth patterns.

Default: 16

5.9.23.1.7 **coil-longitudinal-scale** int or float

Longitudinal scale used when resolving the built-in coil pattern.

Default: 1.25

5.9.23.1.8 **anchor-start** bool

Force the generated path to start on the base path.

Default: true

5.9.23.1.9 anchor-end `bool`

Force the generated path to end on the base path.

Default: `true`

5.9.23.1.10 accuracy `float`

Geometry approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

5.9.24 point

Build a numeric point tuple.

5.9.24.1 Parameters

```
point(  
  x: int float ,  
  y: int float  
) -> array
```

5.9.24.1.1 x `int` or `float`

X coordinate.

5.9.24.1.2 y `int` or `float`

Y coordinate.

5.9.25 points

Return the points visited by a Kurvst path's command stream.

5.9.25.1 Parameters

```
points(path: dictionary) -> array
```

5.9.25.1.1 path `dictionary`

Kurvst path dictionary to inspect.

5.9.26 quad

Build a quadratic path fragment.

5.9.26.1 Parameters

```
quad(  
  start: array ,  
  control: array ,  
  end: array  
) -> dictionary
```

5.9.26.1.1 start array

Start point.

5.9.26.1.2 control array

Quadratic control point.

5.9.26.1.3 end array

Endpoint.

5.9.27 quad-to

Build a quad path element.

5.9.27.1 Parameters

```
quad-to(  
  control: array ,  
  end: array  
) -> dictionary
```

5.9.27.1.1 control array

Quadratic control point.

5.9.27.1.2 end array

Quadratic endpoint.

5.9.28 resolve-length

Resolve a fixed and relative visible path length.

5.9.28.1 Parameters

```
resolve-length(  
    base-length: int float,  
    length: none int float,  
    ratio: none int float,  
    method: string function  
) -> none int float
```

5.9.28.1.1 base-length int or float

Full base path arc length.

5.9.28.1.2 length none or int or float

Fixed target arc length.

Default: none

5.9.28.1.3 ratio none or int or float

Relative target length as a fraction of base-length.

Default: none

5.9.28.1.4 method string or function

Resolution strategy for fixed and relative targets.

Default: "min"

5.9.29 segments

Return drawable cubic segments for any Kurvst path dictionary.

5.9.29.1 Parameters

```
segments(path: dictionary) -> array
```

5.9.29.1.1 path dictionary

Kurvst path dictionary to convert.

5.9.30 split-through

Split a path through a point sequence into per-span paths.

5.9.30.1 Parameters

```
split-through(  
  points: array ,  
  omega: float ,  
  start-outset: int float ,  
  end-outset: int float ,  
  accuracy: float  
) -> dictionary
```

5.9.30.1.1 points array

Two or more points for the curve to pass through.

5.9.30.1.2 omega float

Hobby curl/tension parameter.

Default: 1.0

5.9.30.1.3 start-outset int or float

Arc length removed from the first span start.

Default: 0

5.9.30.1.4 end-outset int or float

Arc length removed from the last span end.

Default: 0

5.9.30.1.5 accuracy float

Geometry approximation accuracy passed to the Rust geometry engine.

Default: 0.001

5.9.31 to-cetz

Draw a path dictionary through CeTZ.

5.9.31.1 Parameters

```
to-cetz(  
  path: dictionary ,  
  unit: int float length ratio ,  
  ..style: any  
) -> content
```

5.9.31.1.1 **path** dictionary

Kurvst path dictionary to draw.

5.9.31.1.2 **unit** int or float or length or ratio

Coordinate multiplier for emitted CeTZ points.

Default: 1

5.9.31.1.3 **..style** any

CeTZ draw style arguments forwarded to merge-path.

5.9.32 **to-cetz-data**

Emit a Kurvst path as CeTZ path data.

5.9.32.1 **Parameters**

```
to-cetz-data(  
  path: dictionary ,  
  unit: int float length ratio  
) -> array
```

5.9.32.1.1 **path** dictionary

Kurvst path dictionary to convert.

5.9.32.1.2 **unit** int or float or length or ratio

Coordinate multiplier for emitted CeTZ points.

Default: 1

5.9.33 **to-native**

Emit a Kurvst path as native Typst curve content.

5.9.33.1 **Parameters**

```
to-native(  
  path: dictionary ,  
  unit: int float length ratio ,  
  ..style: any  
) -> content
```

5.9.33.1.1 **path** dictionary

Kurvst path dictionary to emit.

5.9.33.1.2 **unit** int or float or length or ratio

Coordinate multiplier for emitted Typst curve points.

Default: 1

5.9.33.1.3 **..style** any

Native curve style arguments.

5.9.34 **trim**

Trim a path by arc length from each end.

5.9.34.1 **Parameters**

```
trim(  
  path: dictionary ,  
  start-outset: int float ,  
  end-outset: int float ,  
  accuracy: float  
) -> dictionary
```

5.9.34.1.1 **path** dictionary

Kurvst path dictionary to trim.

5.9.34.1.2 **start-outset** int or float

Arc length removed from the start.

Default: 0

5.9.34.1.3 **end-outset** int or float

Arc length removed from the end.

Default: 0

5.9.34.1.4 accuracy `float`

Arc-length approximation accuracy passed to the Rust geometry engine.

Default: `0.001`

5.9.35 wave

A smooth sinusoidal path pattern.

5.9.35.1 Parameters

`wave(samples-per-period: int) -> dictionary`

5.9.35.1.1 samples-per-period `int`

Number of samples used to approximate one wave period.

Default: `16`

5.9.36 zigzag

A straight-segment triangular path pattern.

5.9.36.1 Parameters

`zigzag()` -> `dictionary`

5.9.37 layer-defaults `dictionary`

Default geometry options for derived path layers.

6 CLI, Rust, and Python APIs

6.1 Rust packages

The Rust API spans two primary packages.

- `gammaloops` contains the physics implementation, data model, integrands, integration, observables, and lower-level algorithms.
- `gammaloop-api` provides the supported state-loading API, session/command integration, CLI assembly, and the PyO3 extension used by the Python distribution.

Load a state with `StateLoadOption::load()`. Its options select a state directory, boot card, logging overrides, read-only behavior, and optional settings overrides. Once loaded, callers can create a CLI-style session or use dedicated structured operations. A raw command string is the compatibility fallback, not a replacement for a typed operation where one exists.

```

use std::path::PathBuf;
use gammaloop_api::{state::CommandHistory, StateLoadOption};

let mut loaded = StateLoadOption {
    state_folder: Some(PathBuf::from("./state")),
    ..StateLoadOption::default()
}.load()?;

let command = CommandHistory::from_raw_string("display settings global"?;
loaded.cli_session().execute_command(command)?;

```

Some Rust types are public so that GammaLoop's packages can work together, but are not intended as stable integration points. Prefer the state-loading and structured-operation APIs described here; start with the curated Rust API and use its Rustdoc links when you need lower-level types or trait implementations.

6.2 Python packaging

A standalone GammaLoop distribution

The Python distribution and import package are both named `gammaloop`. Its compiled backend is `gammaloop._gammaloop`, which applications normally access through the public package. This is separate from the `symbolica.community.*` modules used by Spenso, Idenso, and Vakint; installing GammaLoop does not install those community modules as independent packages.

The main Python entry point is `GammaLoopAPI`. Its constructor loads or creates one stateful session:

```

from gammaloop import GammaLoopAPI

gl = GammaLoopAPI(
    state_folder="./state",
    boot_commands_path="./run.toml",
)
gl.run("display settings global")

```

The Python package requires Python 3.11 or newer. Run just `build-api` when building the bindings from a source checkout.

6.3 Inspect and evaluate an existing state

The following workflow assumes that `./state` contains a generated integrand. Evaluation points are integrand-specific: their length must match the selected integrand and any discrete dimensions. Use the generated reference for the exact `single-sample` and `batch` contracts.

```

from collections.abc import Sequence

from gammaloop import GammaLoopAPI

api = GammaLoopAPI(state_folder="./state", read_only_state=True)
api.run("display processes")

print(api.get_run_history())
runtime = api.get_default_runtime_settings()
print(runtime.get("integrator.n_start"))

def evaluate_one(point: Sequence[float]) -> complex:
    result = api.evaluate_sample(

```

```
    point,  
    process_id=0,  
    minimal_output=True,  
)  
return result.integrand_result
```

Verification tier: compile

The documentation checks compile this example as Python syntax. They do not import or execute the native module: a runtime check additionally needs a built GammaLoop package, a provisioned backend and license, an existing generated state, and a point with the correct dimension.

The `run` method uses the same command language as the CLI and can change the in-memory state, settings, and run history. `read_only_state=True` protects files inside the active state directory; it does not make the Python object immutable or automatically persist the session. Inspect the live session through `get_run_history`, `get_global_settings`, and `get_active_command_blocks`.

For structured inspection, `get_integrand_info` describes the selected backend and graph structure, while `get_integrand_settings` and `get_default_runtime_settings` return detached, read-only `SettingsValue` snapshots. Use `get(path)`, attribute access, indexing, or `to_dict()` to read them; modifying derived Python values does not update the live session.

6.4 Sample evaluation and precision

`evaluate_sample` returns one sample result and the observable bundle for that one-sample batch. `evaluate_samples` accepts a two-dimensional NumPy array and returns per-sample results plus a batch-global observable bundle. Per-sample weights, discrete coordinates, graph names, and orientations must have the same row count as the input batch when provided. Graph and orientation selection applies only to momentum-space evaluation.

Both the ordinary Rust and Python endpoints expose numeric results through an `f64` contract, even when `use_arb_prec=True` requests arbitrary-precision internal evaluation. Evaluation may warm the integrand and update in-memory caches or observable snapshots, including in a read-only-state session. Rust-only callers that must retain the active precision use `evaluate_sample_precise` and `evaluate_samples_precise`.

6.5 Symbolic conversion helpers

The supported module-level helpers are `atom_to_canonical_string` for parsing and canonicalizing a Symbolica expression, `evaluate_graph_overall_factor` for evaluating a graph's symbolic prefactor, and `to_dots` for rewriting tensor contractions into Idenso dot-product notation. Their generated entries document accepted strings, return values, failure modes, and task-oriented examples.

6.6 CLI and settings reference

Use `./gammaloop --help` and the generated CLI and settings reference for command names, aliases, flags, positional arguments, defaults, possible values, and setting paths. The tutorials explain how commands and settings combine into a persistent workflow. The [logging and diagnostics guide](#) covers startup precedence, selective tag filters, sink routing, release-build limitations, and copyable investigation patterns.

7 Releases and change history

GammaLoop versions are identified by repository tags together with the version of the Python distribution and the `gammaloops` Rust package. These components can evolve independently, so record all of them when reporting a result or asking for support.

7.1 Choosing the matching documentation

The `latest` channel follows current development. Documentation under `snapshots/<tag>` stays with a tagged release. Use the snapshot matching your checkout whenever command options, state files, or API signatures differ from `latest`.

7.2 Before upgrading

Release notes currently live with repository tags and their linked pull requests rather than in one consolidated changelog. Review them before upgrading, especially when you depend on persisted states, CLI scripts, Python bindings, or numerical results. Keep a copy of valuable state directories and re-run validation samples after changing versions.

Information to include in a report

Include the repository tag or commit, the Python distribution version, the `gammalooprs` version, the run card, and any relevant settings overrides. This makes a calculation easier to reproduce and distinguishes a software change from a configuration change.

API reference

The packages available in this version are listed below. Follow an item link for its complete language reference.

gammaLooprs

GammaLoop's supported Rust entry points. The complete compiled surface remains available in the Rustdoc sidecar.

GammaLoopContext

Combines the state-map and model capabilities required by GammaLoop computations.

Combines the state-map and model capabilities required by GammaLoop computations.

```
pub trait GammaLoopContext : HasStateMap + HasModel {}
```

Supported usage

```
fn accepts_context<C: gammaLooprs::GammaLoopContext>(_context: &C) {}
```

[Exhaustive reference](#) · [Source](#)

HasModel

Provides shared access to the physics model owned by a computation context.

Provides shared access to the physics model owned by a computation context.

```
pub trait HasModel { fn get_model(& self) -> & Model; }
```

Supported usage

```
fn model_owner<C: gammaLooprs::HasModel>(_context: &C) {}
```

get_model

Kind: Method

```
fn get_model(& self) -> & Model;
```

[Exhaustive reference](#) · [Source](#)

Install interrupt handling

Installs GammaLoop's process interrupt handler before a long-running integration.

Installs GammaLoop's process interrupt handler before a long-running integration.

```
fn set_interrupt_handler()
```

Supported usage

```
gammaLooprs::set_interrupt_handler();
```

[Exhaustive reference](#) · [Source](#)

gammaLoop-api

GammaLoop's stable facade and command API. Use this curated boundary for applications while consulting the full Rustdoc for exhaustive details.

State loading options

Selects the state folder, boot commands, settings overrides, logging, and read-only mode.

Selects the state folder, boot commands, settings overrides, logging, and read-only mode.


```
#[derive(Debug, Clone, PartialEq, Default)] pub struct StateLoadOption
{
    pub clean_state : bool, pub boot_commands_path : Option < PathBuf > , pub
    state_folder : Option < PathBuf > , pub model_file : Option < PathBuf > ,
    pub trace_logs_filename : Option < String > , pub level : Option <
    LogLevel > , pub logfile_level : Option < LogLevel > , pub logging_prefix
    : Option < LogFormat > , pub read_only_state : bool, pub
    settings_global_path : Option < PathBuf > , pub
    settings_runtime_defaults_path : Option < PathBuf > ,
}
```

Reference feature matrix: cli (item is available without an additional gate)

Supported usage

```
let options = gammaloop_api::StateLoadOption::default();
```

clean_state

Kind: Field

bool

boot_commands_path

Kind: Field

Option < PathBuf >

state_folder

Kind: Field

Option < PathBuf >

model_file

Kind: Field

Option < PathBuf >

trace_logs_filename

Kind: Field

Option < String >

level

Kind: Field

Option < LogLevel >

logfile_level

Kind: Field

Option < LogLevel >

logging_prefix

Kind: Field

Option < LogFormat >

read_only_state

Kind: Field

bool

settings_global_path

Kind: Field

Option < PathBuf >

settings_runtime_defaults_path

Kind: Field

Option < PathBuf >

Exhaustive reference · Source

Load a GammaLoop state

Loads the selected state and returns the supported session boundary.

Loads the selected state and returns the supported session boundary.

```
fn load(self) -> Result < LoadedState >
```

Reference feature matrix: cli (item is available without an additional gate)

Parameter	Type
self	self

Returns: Result < LoadedState >

Supported usage

```
let loaded = gammaloop_api::StateLoadOption::default().load()?;
```

Exhaustive reference · Source

Create a CLI session

Borrows a loaded state as a command session while keeping state ownership explicit.

Borrows a loaded state as a command session while keeping state ownership explicit.

```
fn cli_session(& mut self) -> CliSession < '_ >
```

Reference feature matrix: cli (item is available without an additional gate)

Parameter	Type
self	& mut self

Returns: CliSession < '_ >

Supported usage

```
let mut loaded = gammaloop_api::StateLoadOption::default().load()?; let session =  
loaded.cli_session();
```

Exhaustive reference · Source

Command-line settings

Owns the generated command-line, global, session, and state-loading settings.

Owns the generated command-line, global, session, and state-loading settings.

```
#[derive(Debug, Clone, Deserialize, Serialize, JsonSchema)]  
#[serde(default, deny_unknown_fields)] pub struct CLISettings  
{
```

```
    #[doc =
```

```
        " Serialize replayable commands as compact strings when their structured form
```

```

permits it."]
    #[serde(skip_serializing_if = "is_true")] pub try_strings : bool,
    #[doc =
    " Permit generated artifacts to replace compatible files in the active state
folder." ]
    #[serde(skip_serializing_if = "is_false")] pub override_state : bool,
    #[doc =
    " Location and optional display name of the persistent GammaLoop state." ]
    #[serde(skip_serializing_if = "IsDefault::is_default")] pub state :
    StateSettings,
    #[doc =
    " Generation, logging, and parallelization settings shared by the CLI session." ]
    #[serde(skip_serializing_if = "IsDefault::is_default")] pub global :
    GlobalSettings, #[serde(skip)] #[schemars(skip)] pub session :
    SessionSettings,
}

```

Reference feature matrix: cli (item is available without an additional gate)

Supported usage

```
let settings = gammaloop_api::CLISettings::default();
```

try_strings

Kind: Field

bool

Serialize replayable commands as compact strings when their structured form permits it.

override_state

Kind: Field

bool

Permit generated artifacts to replace compatible files in the active state folder.

state

Kind: Field

StateSettings

Location and optional display name of the persistent GammaLoop state.

global

Kind: Field

GlobalSettings

Generation, logging, and parallelization settings shared by the CLI session.

session

Kind: Field

SessionSettings

Exhaustive reference · Source

GammaLoop command

Runs the Clap-derived command tree for state creation, inspection, generation, and evaluation.

Runs the Clap-derived command tree for state creation, inspection, generation, and evaluation.

`gammaloop [OPTIONS] [COMMAND]`

Reference feature matrix: cli (item is available without an additional gate)

Supported usage

`gammaloop --help`

try_strings

Kind: Field

bool

Serialize replayable commands as compact strings when their structured form permits it.

override_state

Kind: Field

bool

Permit generated artifacts to replace compatible files in the active state folder.

state

Kind: Field

StateSettings

Location and optional display name of the persistent GammaLoop state.

global

Kind: Field

GlobalSettings

Generation, logging, and parallelization settings shared by the CLI session.

session

Kind: Field

SessionSettings

Exhaustive reference · Source

gammaloop-python

Exports registered by gammaloop.

GammaLoopAPI

Load, inspect, and evaluate one mutable GammaLoop session.

Load, inspect, and evaluate one mutable GammaLoop session.

One instance owns a GammaLoop state, run history, CLI settings, default runtime settings, and session state. Calls to ``run`` and the evaluation methods all act on that same in-memory session.

Notes

``read\_only\_state=True`` prevents writes inside the active state directory; it does not make this Python object immutable. Commands may still change in-memory

settings, processes, or run history. Create separate instances when independent sessions are required.

class GammaLoopAPI:

Requires features: python_api, python_abi, python_stubgen

Introspection scaffold (not a task example)

```
from gammaloop._gammaloop import GammaLoopAPI
help(GammaLoopAPI)
```

__new__

Kind: AssociatedFunction

```
def __new__(cls, state_folder: typing.Optional[builtins.str | os.PathLike |
pathlib.Path] = None, boot_commands_path: typing.Optional[builtins.str | os.PathLike
| pathlib.Path] = None, model_file: typing.Optional[builtins.str | os.PathLike |
pathlib.Path] = None, trace_logs_filename: typing.Optional[builtins.str] = None,
level: typing.Optional[LogLevel] = None, logfile_level: typing.Optional[LogLevel] =
None, logging_prefix: builtins.object | None = None, read_only_state: builtins.bool
= ..., settings_global_path: typing.Optional[builtins.str | os.PathLike |
pathlib.Path] = None, settings_runtime_defaults_path: typing.Optional[builtins.str |
os.PathLike | pathlib.Path] = None, clean_state: builtins.bool = ...) ->
GammaLoopAPI:
```

Load or create a GammaLoop state and initialize its CLI session.

Parameters

state_folder : path-like, optional

State directory to load or create. The default is ``./gammaloop\_state``.

boot_commands_path : path-like, optional

TOML run card whose commands are applied during startup.

model_file : path-like, optional

Model file override used while loading or initializing the state.

trace_logs_filename : str, optional

File receiving native trace records for this session.

level : LogLevel, optional

Terminal log-level override for this session.

logfile_level : LogLevel, optional

File log-level override for this session.

logging_prefix : object, optional

Native logging-prefix configuration.

read_only_state : bool, default=False

Prevent writes whose target lies inside the active state directory and disable file logging there. In-memory session changes remain possible.

settings_global_path : path-like, optional

TOML file overriding the global settings loaded at startup.

settings_runtime_defaults_path : path-like, optional

TOML file overriding the default runtime settings loaded at startup.

clean_state : bool, default=False

Remove the resolved state path before startup. This is destructive and cannot be combined with ``read\_only\_state=True``.

Returns

GammaLoopAPI

A stateful API instance sharing one state, history, and settings session.

Raises

Exception

If startup options conflict, the state or settings cannot be loaded, a boot command fails, or a boot card requests process exit.

Examples

Open an existing generated state without permitting writes to it::

```
api = GammaLoopAPI(state_folder="./state", read_only_state=True)
```

state_folder

Kind: Parameter

state_folder: typing.Optional[builtins.str | os.PathLike | pathlib.Path] = None

Default: None

State directory to load or create. The default is ``./gammaloop\_state``.

boot_commands_path

Kind: Parameter

boot_commands_path: typing.Optional[builtins.str | os.PathLike | pathlib.Path] = None

Default: None

TOML run card whose commands are applied during startup.

model_file

Kind: Parameter

model_file: typing.Optional[builtins.str | os.PathLike | pathlib.Path] = None

Default: None

Model file override used while loading or initializing the state.

trace_logs_filename

Kind: Parameter

trace_logs_filename: typing.Optional[builtins.str] = None

Default: None

File receiving native trace records for this session.

level

Kind: Parameter

level: typing.Optional[LogLevel] = None

Default: None

Terminal log-level override for this session.

logfile_level

Kind: Parameter

logfile_level: typing.Optional[LogLevel] = None

Default: None

File log-level override for this session.

logging_prefix

Kind: Parameter

logging_prefix: builtins.object | None = None

Default: None

Native logging-prefix configuration.

read_only_state

Kind: Parameter

read_only_state: builtins.bool = ...

Default: ...

Prevent writes whose target lies inside the active state directory and disable file logging there. In-memory session changes remain possible.

settings_global_path

Kind: Parameter

settings_global_path: typing.Optional[builtins.str | os.PathLike | pathlib.Path] = None

Default: None

TOML file overriding the global settings loaded at startup.

settings_runtime_defaults_path

Kind: Parameter

settings_runtime_defaults_path: typing.Optional[builtins.str | os.PathLike | pathlib.Path] = None

Default: None

TOML file overriding the default runtime settings loaded at startup.

clean_state

Kind: Parameter

clean_state: builtins.bool = ...

Default: ...

Remove the resolved state path before startup. This is destructive and cannot be combined with ``read\_only\_state=True``.

evaluate_sample

Kind: Method

```
def evaluate_sample(self, point: typing.Sequence[builtins.float], process_id:
typing.Optional[builtins.int] = None, integrand_name: typing.Optional[builtins.str] =
None, use_arb_prec: builtins.bool = ..., minimal_output: builtins.bool = ...,
return_events: typing.Optional[builtins.bool] = None, momentum_space: builtins.bool
= ..., integrator_weight: typing.Optional[builtins.float] = None, discrete_dim:
typing.Optional[typing.Sequence[builtins.int]] = None, graph_name:
```

```
typing.Optional[builtins.str] = None, orientation: typing.Optional[builtins.int] =
None) -> EvaluationResult:
```

Evaluate one integration or momentum-space sample.

Parameters

point : Sequence[float]

Coordinates for one sample. In integration space, the length must match the selected integrand and ``discrete\_dim``. In momentum space, values are grouped as ``(px, py, pz)`` and the length must be a multiple of 3.

process_id : int, optional

Process containing the integrand. Supply this when selection is ambiguous.

integrand_name : str, optional

Integrand to evaluate. Supply this when selection is ambiguous.

use_arb_prec : bool, default=False

Request arbitrary-precision internal evaluation.

minimal_output : bool, default=False

Omit the optional evaluation metadata from the returned sample.

return_events : bool, optional

Temporarily override event generation for this call. The integrand setting is restored afterward.

momentum_space : bool, default=False

Interpret ``point`` as flattened three-momenta instead of integration-space coordinates.

integrator_weight : float, optional

Weight associated with this sample. The default is 1.0.

discrete_dim : Sequence[int], optional

Discrete integration coordinates used to determine the expected dimension.

graph_name : str, optional

Graph selected for momentum-space evaluation.

orientation : int, optional

Orientation index for ``graph\_name`` in momentum-space evaluation.

Returns

EvaluationResult

The sample evaluation and the observable snapshot for its one-row batch.

Raises

Exception

If integrand selection is ambiguous or invalid, dimensions do not match, graph or orientation selection is invalid, or warm-up/evaluation fails.

Notes

Arbitrary precision affects the internal calculation, but Python-visible numeric fields use the package's ``float64`` output contract. Evaluation may warm the integrand and update in-memory caches or observable snapshots even in a read-only-state session.

Examples

Evaluate a point whose dimension matches integrand 0::


```
result = api.evaluate_sample(point, process_id=0, minimal_output=True)
value = result.integrand_result
```

point

Kind: Parameter

point: typing.Sequence[builtins.float]

Coordinates for one sample. In integration space, the length must match the selected integrand and ``discrete\_dim``. In momentum space, values are grouped as ``(px, py, pz)`` and the length must be a multiple of 3.

process_id

Kind: Parameter

process_id: typing.Optional[builtins.int] = None

Default: None

Process containing the integrand. Supply this when selection is ambiguous.

integrand_name

Kind: Parameter

integrand_name: typing.Optional[builtins.str] = None

Default: None

Integrand to evaluate. Supply this when selection is ambiguous.

use_arb_prec

Kind: Parameter

use_arb_prec: builtins.bool = ...

Default: ...

Request arbitrary-precision internal evaluation.

minimal_output

Kind: Parameter

minimal_output: builtins.bool = ...

Default: ...

Omit the optional evaluation metadata from the returned sample.

return_events

Kind: Parameter

return_events: typing.Optional[builtins.bool] = None

Default: None

Temporarily override event generation for this call. The integrand setting is restored afterward.

momentum_space

Kind: Parameter

momentum_space: builtins.bool = ...

Default: ...

Interpret ``point`` as flattened three-momenta instead of integration-space coordinates.

integrator_weight

Kind: Parameter

integrator_weight: typing.Optional[builtins.float] = None

Default: None

Weight associated with this sample. The default is 1.0.

discrete_dim

Kind: Parameter

discrete_dim: typing.Optional[typing.Sequence[builtins.int]] = None

Default: None

Discrete integration coordinates used to determine the expected dimension.

graph_name

Kind: Parameter

graph_name: typing.Optional[builtins.str] = None

Default: None

Graph selected for momentum-space evaluation.

orientation

Kind: Parameter

orientation: typing.Optional[builtins.int] = None

Default: None

Orientation index for ``graph\_name`` in momentum-space evaluation.

evaluate_samples

Kind: Method

```
def evaluate_samples(self, points: numpy.typing.NDArray[numpy.float64], process_id:
typing.Optional[builtins.int] = None, integrand_name: typing.Optional[builtins.str] =
None, use_arb_prec: builtins.bool = ..., minimal_output: builtins.bool = ...,
return_events: typing.Optional[builtins.bool] = None, momentum_space: builtins.bool
= ..., integrator_weights: typing.Optional[numpy.typing.NDArray[numpy.float64]] =
None, discrete_dims: numpy.typing.NDArray[numpy.unsignedinteger] | None = None,
graph_names: typing.Optional[typing.Sequence[typing.Optional[builtins.str]]] = None,
orientations: typing.Optional[typing.Sequence[typing.Optional[builtins.int]]] = None)
-> BatchEvaluationResult:
```

Evaluate a batch of integration or momentum-space samples.

Parameters

points : numpy.ndarray[numpy.float64]

Two-dimensional array with one sample per row. Integration-space columns must match the selected integrand; momentum-space columns are flattened ``(px, py, pz)`` groups.

process_id : int, optional
 Process containing the integrand. Supply this when selection is ambiguous.
integrand_name : str, optional
 Integrand to evaluate. Supply this when selection is ambiguous.
use_arb_prec : bool, default=False
 Request arbitrary-precision internal evaluation.
minimal_output : bool, default=False
 Omit the optional evaluation metadata from every returned sample.
return_events : bool, optional
 Temporarily override event generation for this call. The integrand setting is restored afterward.
momentum_space : bool, default=False
 Interpret each row as flattened three-momenta instead of integration-space coordinates.
integrator_weights : numpy.ndarray[numpy.float64], optional
 One weight per row. Defaults to 1.0 for every sample.
discrete_dims : numpy.ndarray[numpy.unsignedinteger], optional
 Two-dimensional array with one row of discrete coordinates per sample.
graph_names : Sequence[str | None], optional
 Momentum-space graph selection for each sample.
orientations : Sequence[int | None], optional
 Momentum-space orientation selection for each sample.

Returns

BatchEvaluationResult

Per-sample evaluations and one observable snapshot for the complete batch.

Raises

Exception

If integrand selection is ambiguous or invalid, array or option lengths do not match, dimensions are invalid, or warm-up/evaluation fails.

Notes

Arbitrary precision affects the internal calculation, but Python-visible numeric fields use the package's ``float64`` output contract. Evaluation may update in-memory caches or observable snapshots in a read-only-state session.

Examples

Evaluate a caller-provided two-dimensional array::

```
points = numpy.asarray(points, dtype=numpy.float64)
result = api.evaluate_samples(points, process_id=0, minimal_output=True)
```

points

Kind: Parameter

points: numpy.typing.NDArray[numpy.float64]

Two-dimensional array with one sample per row. Integration-space columns must match the selected integrand; momentum-space columns are flattened ``(px, py, pz)`` groups.

process_id

Kind: Parameter

`process_id: typing.Optional[builtins.int] = None`

Default: None

Process containing the integrand. Supply this when selection is ambiguous.

integrand_name

Kind: Parameter

`integrand_name: typing.Optional[builtins.str] = None`

Default: None

Integrand to evaluate. Supply this when selection is ambiguous.

use_arb_prec

Kind: Parameter

`use_arb_prec: builtins.bool = ...`

Default: ...

Request arbitrary-precision internal evaluation.

minimal_output

Kind: Parameter

`minimal_output: builtins.bool = ...`

Default: ...

Omit the optional evaluation metadata from every returned sample.

return_events

Kind: Parameter

`return_events: typing.Optional[builtins.bool] = None`

Default: None

Temporarily override event generation for this call. The integrand setting is restored afterward.

momentum_space

Kind: Parameter

`momentum_space: builtins.bool = ...`

Default: ...

Interpret each row as flattened three-momenta instead of integration-space coordinates.

integrator_weights

Kind: Parameter

`integrator_weights: typing.Optional[numpy.typing.NDArray[numpy.float64]] = None`

Default: None

One weight per row. Defaults to 1.0 for every sample.

discrete_dims

Kind: Parameter

discrete_dims: numpy.typing.NDArray[numpy.unsignedinteger] | None = None

Default: None

Two-dimensional array with one row of discrete coordinates per sample.

graph_names

Kind: Parameter

graph_names: typing.Optional[typing.Sequence[typing.Optional[builtins.str]]] = None

Default: None

Momentum-space graph selection for each sample.

orientations

Kind: Parameter

orientations: typing.Optional[typing.Sequence[typing.Optional[builtins.int]]] = None

Default: None

Momentum-space orientation selection for each sample.

import_graphs

Kind: Method

```
def import_graphs(self, graphs: builtins.str, process_name:
typing.Optional[builtins.str] = None, process_id: typing.Optional[builtins.int] =
None, integrand_name: typing.Optional[builtins.str] = None, format: builtins.str
= ..., overwrite: builtins.bool = ..., append: builtins.bool = ...) -> None:
```

graphs

Kind: Parameter

graphs: builtins.str

process_name

Kind: Parameter

process_name: typing.Optional[builtins.str] = None

Default: None

process_id

Kind: Parameter

process_id: typing.Optional[builtins.int] = None

Default: None

integrand_name

Kind: Parameter

integrand_name: typing.Optional[builtins.str] = None

Default: None

format

Kind: Parameter

format: builtins.str = ...

Default: ...

overwrite

Kind: Parameter

overwrite: builtins.bool = ...

Default: ...

append

Kind: Parameter

append: builtins.bool = ...

Default: ...

get_lmbs

Kind: Method

```
def get_lmbs(self, graphs: builtins.str, format: builtins.str = ...) ->
builtins.list[builtins.list[tuple[builtins.list[builtins.int],
builtins.list[builtins.int], builtins.dict[builtins.int,
tuple[builtins.list[builtins.int], builtins.list[builtins.int]]]]]:
```

graphs

Kind: Parameter

graphs: builtins.str

format

Kind: Parameter

format: builtins.str = ...

Default: ...

get_orientations

Kind: Method

```
def get_orientations(self, graph_name: builtins.str, process_id:
typing.Optional[builtins.int] = None, integrand_name: typing.Optional[builtins.str] =
None) -> builtins.list[builtins.dict[builtins.int, builtins.int]]:
```

graph_name

Kind: Parameter

graph_name: builtins.str

process_id

Kind: Parameter

process_id: typing.Optional[builtins.int] = None

Default: None

integrand_name

Kind: Parameter

integrand_name: typing.Optional[builtins.str] = None

Default: None

get_model**Kind:** Method

```
def get_model(self) -> builtins.str:
```

evaluate**Kind:** Method

```
def evaluate(self, process_id: typing.Optional[builtins.int] = None,
graphs_group_name: typing.Optional[builtins.str] = None, result_path:
typing.Optional[builtins.str | os.PathLike | pathlib.Path] = None, numerical:
builtins.bool = ..., number_of_terms_in_epsilon_expansion:
typing.Optional[builtins.int] = None) -> builtins.str:
```

process_id**Kind:** Parameter

```
process_id: typing.Optional[builtins.int] = None
```

Default: None**graphs_group_name****Kind:** Parameter

```
graphs_group_name: typing.Optional[builtins.str] = None
```

Default: None**result_path****Kind:** Parameter

```
result_path: typing.Optional[builtins.str | os.PathLike | pathlib.Path] = None
```

Default: None**numerical****Kind:** Parameter

```
numerical: builtins.bool = ...
```

Default: ...**number_of_terms_in_epsilon_expansion****Kind:** Parameter

```
number_of_terms_in_epsilon_expansion: typing.Optional[builtins.int] = None
```

Default: None**import_model****Kind:** Method

```
def import_model(self, model_specifier: builtins.str | os.PathLike | pathlib.Path,
simplify_model: builtins.bool = ...) -> None:
```

model_specifier**Kind:** Parameter

```
model_specifier: builtins.str | os.PathLike | pathlib.Path
```

simplify_model

Kind: Parameter

simplify_model: builtins.bool = ...

Default: ...

list_outputs

Kind: Method

```
def list_outputs(self) -> tuple[builtins.dict[builtins.str, builtins.int],
builtins.dict[builtins.str, builtins.int]]:
```

get_integrand_info

Kind: Method

```
def get_integrand_info(self, process_id: typing.Optional[builtins.int] = None,
integrand_name: typing.Optional[builtins.str] = None) -> IntegrandInfo:
```

Describe the selected generated integrand and its graph structure.

Parameters

process_id : int, optional

Process containing the integrand. Supply this when selection is ambiguous.

integrand_name : str, optional

Integrand to inspect. Supply this when selection is ambiguous.

Returns

IntegrandInfo

Structured process, backend, graph, orientation, cut, and size metadata.

Raises

Exception

If no unique generated integrand matches the selection.

Examples

Inspect the active evaluation backend and graph count::

```
info = api.get_integrand_info(process_id=0)
print(info.active_f64_backend, info.graph_count)
```

process_id

Kind: Parameter

process_id: typing.Optional[builtins.int] = None

Default: None

Process containing the integrand. Supply this when selection is ambiguous.

integrand_name

Kind: Parameter

integrand_name: typing.Optional[builtins.str] = None

Default: None

Integrand to inspect. Supply this when selection is ambiguous.

get_integrand_settings

Kind: Method

```
def get_integrand_settings(self, process_id: typing.Optional[builtins.int] = None,
integrand_name: typing.Optional[builtins.str] = None) -> SettingsValue:
```

Return a detached, read-only snapshot of one integrand's settings.

Parameters

process_id : int, optional

Process containing the integrand. Supply this when selection is ambiguous.

integrand_name : str, optional

Integrand whose settings are required.

Returns

SettingsValue

Serialized settings snapshot supporting ``get(path)``, attribute access, indexing, and ``to\_dict()``. Mutating it does not update the live session.

Raises

Exception

If selection fails, the integrand has not been generated, or its settings cannot be serialized.

Examples

Read a nested setting without changing the live integrand::

```
settings = api.get_integrand_settings(process_id=0)
print(settings.get("general.generate_events"))
```

process_id

Kind: Parameter

process_id: typing.Optional[builtins.int] = None

Default: None

Process containing the integrand. Supply this when selection is ambiguous.

integrand_name

Kind: Parameter

integrand_name: typing.Optional[builtins.str] = None

Default: None

Integrand whose settings are required.

get_run_history

Kind: Method

```
def get_run_history(self) -> builtins.str:
```

Render the current in-memory run history as TOML.

Returns

str

TOML representation of the history owned by this API instance.

Raises

Exception

If the current history cannot be serialized.

Notes

This reports the live session, including commands run through ``run``; it does not imply that the history has been persisted to the state directory.

Examples

Capture a reproducible run card from the current session::

```
run_card_toml = api.get_run_history()
```

get_global_settings

Kind: Method

```
def get_global_settings(self) -> builtins.str:
```

Render the current effective CLI and global settings as TOML.

Returns

str

TOML representation of the settings used by this API session.

Raises

Exception

If the settings cannot be rendered.

Examples

Record the effective settings after applying startup overrides::

```
settings_toml = api.get_global_settings()
```

get_active_command_blocks

Kind: Method

```
def get_active_command_blocks(self) -> builtins.dict[builtins.str,  
builtins.list[builtins.str]]:
```

Return the named command blocks in the current run history.

Returns

dict[str, list[str]]

Mapping from block name to its rendered CLI commands.

Examples

Inspect the commands currently grouped under each block::

```
for name, commands in api.get_active_command_blocks().items():
    print(name, commands)
```

get_default_runtime_settings

Kind: Method

```
def get_default_runtime_settings(self) -> SettingsValue:
```

Return a detached, read-only snapshot of the default runtime settings.

Returns

SettingsValue

Serialized settings including defaults. Use ``get(path)`` or ``to\_dict()`` to inspect it; changes to derived Python values do not affect the session.

Raises

Exception

If the settings cannot be serialized.

Examples

Inspect a runtime default by its documented settings path::

```
runtime = api.get_default_runtime_settings()
print(runtime.get("integrator.n_start"))
```

get_dot_files

Kind: Method

```
def get_dot_files(self, process: typing.Optional[builtins.int | builtins.str] = None,
    integrand_name: typing.Optional[builtins.str] = None, settings: DotExportSettings
    = ...) -> builtins.str:
```

process

Kind: Parameter

process: typing.Optional[builtins.int | builtins.str] = None

Default: None

integrand_name

Kind: Parameter

integrand_name: typing.Optional[builtins.str] = None

Default: None

settings

Kind: Parameter

settings: DotExportSettings = ...

Default: ...

run

Kind: Method

```
def run(self, command: builtins.str) -> None:
```

Parse and execute a CLI command in this API instance's session.

Parameters

command : str

GammaLoop CLI command text.

Returns

None

The command's normal output is emitted through the configured CLI/logging sinks; inspect structured state through the corresponding getter methods.

Raises

Exception

If the command cannot be parsed or execution fails.

Notes

Commands share and may mutate the instance's in-memory state, settings, and run history. ``read\_only\_state=True`` only blocks writes inside the active state directory. The API does not automatically persist the session after this call; persistence depends on the executed command's explicit output behavior.

Examples

Display the processes loaded in the current state::

```
api.run("display processes")
```

command

Kind: Parameter

command: builtins.str

GammaLoop CLI command text.

generate_cff

Kind: Method

```
def generate_cff(self, dot_string: builtins.str, subgraph_nodes:
typing.Sequence[builtins.str], reverse_dangling: typing.Sequence[builtins.int],
orientation_pattern: typing.Optional[builtins.str] = None) ->
builtins.list[tuple[builtins.dict[builtins.int, builtins.int], builtins.str]]:
```

dot_string

Kind: Parameter

dot_string: builtins.str

subgraph_nodes

Kind: Parameter

subgraph_nodes: typing.Sequence[builtins.str]

reverse_dangling

Kind: Parameter

reverse_dangling: typing.Sequence[builtins.int]

orientation_pattern

Kind: Parameter

orientation_pattern: typing.Optional[builtins.str] = None

Default: None

generate_cff_as_json_string

Kind: Method

```
def generate_cff_as_json_string(self, dot_string: builtins.str, subgraph_nodes:
typing.Sequence[builtins.str], reverse_dangling: typing.Sequence[builtins.int],
orientation_pattern: typing.Optional[builtins.str] = None) -> builtins.str:
```

dot_string

Kind: Parameter

dot_string: builtins.str

subgraph_nodes

Kind: Parameter

subgraph_nodes: typing.Sequence[builtins.str]

reverse_dangling

Kind: Parameter

reverse_dangling: typing.Sequence[builtins.int]

orientation_pattern

Kind: Parameter

orientation_pattern: typing.Optional[builtins.str] = None

Default: None

Exhaustive reference · Source

atom_to_canonical_string

Parse a Symbolica expression and return its canonical string form.

Parse a Symbolica expression and return its canonical string form.

Parameters

atom_str : str

Symbolica expression to parse and normalize.

Returns

str

The parsed expression in canonical string form.

Raises

Exception

If ``atom\_str`` is not a valid Symbolica expression.

Examples

Normalize a symbolic expression before comparing or storing it::

```
atom_to_canonical_string("x + x")
```

```
def atom_to_canonical_string(atom_str: builtins.str) -> builtins.str:
```

Requires features: python_api, python_abi, python_stubgen

Introspection scaffold (not a task example)

```
from gammaloop._gammaloop import atom_to_canonical_string
help(atom_to_canonical_string)
```

Exhaustive reference · Source

evaluate_graph_overall_factor

Evaluate a graph's symbolic overall factor and return its canonical form.

Evaluate a graph's symbolic overall factor and return its canonical form.

Parameters

overall_factor : str

Symbolica expression used as the graph's overall factor.

Returns

str

The evaluated factor in canonical Symbolica form.

Raises

Exception

If the expression cannot be parsed or evaluated.

Examples

Evaluate the symmetry and fermion-loop factors attached to a graph::

```
evaluate_graph_overall_factor(
    "AutG(2)^-1*InternalFermionLoopSign(-1)"
)
```

```
def evaluate_graph_overall_factor(overall_factor: builtins.str) -> builtins.str:
```

Requires features: python_api, python_abi, python_stubgen

Introspection scaffold (not a task example)

```
from gammaloop._gammaloop import evaluate_graph_overall_factor
help(evaluate_graph_overall_factor)
```

Exhaustive reference · Source

to_dots

Rewrite a Symbolica tensor expression into Idenso dot-product notation.

Rewrite a Symbolica tensor expression into Idenso dot-product notation.

Parameters

atom_str : str

Symbolica tensor expression whose repeated indices encode contractions.

Returns

str

The equivalent expression using Idenso dot-product notation.

Raises

Exception

If ``atom\_str`` cannot be parsed or converted.

Examples

Rewrite a contraction before passing it to an Idenso workflow::

```
to_dots("p(mu) * q(mu)")
```

```
def to_dots(atom_str: builtins.str) -> builtins.str:
```

Requires features: python_api, python_abi, python_stubgen

Introspection scaffold (not a task example)

```
from gammaloop._gammaloop import to_dots
```

```
help(to_dots)
```

Exhaustive reference · Source

CLI commands and settings

Commands and settings available in this version are listed below.

Command tree

Command	Description
gammaLoop	An implementation of the Local Unitarity method for computing differential collider cross-sections.
gammaLoop display	Inspect models, processes, integrands, settings, and named session data
gammaLoop display model	Show model particles, parameters, couplings, and vertices for the selected context
gammaLoop display processes	List processes currently stored in the active state
gammaLoop display integrand	Show generated integrands, graph groups, resources, and selected detail tables
gammaLoop display quantities	List configured named quantities, or inspect one quantity in a selected process
gammaLoop display observables	List configured observables, or inspect one observable in a selected process
gammaLoop display selectors	List configured event selectors, or inspect one selector in a selected process
gammaLoop display command_block	Show one stored command block, or list every block when no name is supplied
gammaLoop display settings	Show effective global, default-runtime, or process-specific settings
gammaLoop display settings global	Show effective global generation, logging, and parallelization settings
gammaLoop display settings default-runtime	Show the default runtime settings inherited by newly generated integrands
gammaLoop display settings process	Show effective settings for one generated process or integrand
gammaLoop duplicate	Copy generated data to a new process or integrand name
gammaLoop duplicate integrand	Copy one generated integrand and its process context under new names
gammaLoop set	Change global, runtime, model, or process settings
gammaLoop set base-dir	Set the base directory for gammaloop state and logs
gammaLoop set global	Change global generation, logging, and parallelism settings
gammaLoop set global file	Merge a TOML file into the current global or default-runtime settings
gammaLoop set global string	Load settings from TOML content passed as a CLI string
gammaLoop set global kv	Merge one or more dotted `KEY=VALUE` assignments into the current settings

Command	Description
gammaLoop set global defaults	Sync process runtime settings from the current default runtime settings
gammaLoop set global stored	Reload the corresponding settings from the active state's stored file
gammaLoop set default-runtime	Change the runtime settings inherited by newly generated integrands
gammaLoop set default-runtime file	Merge a TOML file into the current global or default-runtime settings
gammaLoop set default-runtime string	Load settings from TOML content passed as a CLI string
gammaLoop set default-runtime kv	Merge one or more dotted `KEY=VALUE` assignments into the current settings
gammaLoop set default-runtime defaults	Sync process runtime settings from the current default runtime settings
gammaLoop set default-runtime stored	Reload the corresponding settings from the active state's stored file
gammaLoop set model	Assign model parameters in the selected context or reset them to defaults
gammaLoop set process	Change runtime settings or named analysis objects for one process or integrand
gammaLoop set process file	Merge a TOML file into the selected process or integrand settings
gammaLoop set process string	Load settings from TOML content passed as a CLI string
gammaLoop set process kv	Merge one or more dotted `KEY=VALUE` assignments into the selected settings
gammaLoop set process defaults	Sync process runtime settings from the current default runtime settings
gammaLoop set process stored	Reload the selected process or integrand settings from its stored file
gammaLoop set process add	Add a named quantity/observable/selector
gammaLoop set process add quantity	Add a named derived quantity to the selected process settings
gammaLoop set process add observable	Add a named observable to the selected process settings
gammaLoop set process add selector	Add a named event selector to the selected process settings
gammaLoop set process update	Update a named quantity/observable/selector
gammaLoop set process update quantity	Update fields of an existing named quantity
gammaLoop set process update observable	Update fields of an existing named observable
gammaLoop set process update selector	Update fields of an existing named selector
gammaLoop set process remove	Remove a named quantity/observable/selector

Command	Description
<code>gammaLoop set process remove quantity</code>	Remove a named quantity from the selected process settings
<code>gammaLoop set process remove observable</code>	Remove a named observable from the selected process settings
<code>gammaLoop set process remove selector</code>	Remove a named selector from the selected process settings
<code>gammaLoop import</code>	Import models or previously generated graphs into the active state
<code>gammaLoop import model</code>	Load a UFO model and make it the active model for subsequent generation
<code>gammaLoop import graphs</code>	Load serialized graph data into a new or existing process
<code>gammaLoop save</code>	Export graph data, standalone evaluators, schemas, or the current state
<code>gammaLoop save dot</code>	Export generated graphs as DOT files and drawing templates
<code>gammaLoop save uv-forest</code>	Export the ultraviolet forest for selected generated graphs
<code>gammaLoop save standalone</code>	Export an integrand as a standalone evaluator or serialized data file
<code>gammaLoop save state</code>	Persist the active state, settings, and replayable run history
<code>gammaLoop save schema</code>	Regenerate the JSON schema files for CLI and runtime configuration
<code>gammaLoop run</code>	Execute a stored command block or an inline command list
<code>gammaLoop start_commands_block</code>	Begin recording subsequent commands under a reusable block name
<code>gammaLoop finish_commands_block</code>	Stop recording commands and store the active command block
<code>gammaLoop remove</code>	Remove generated processes from the active state
<code>gammaLoop remove processes</code>	Remove one integrand, one process, or all processes selected by the supplied target
<code>gammaLoop integrate</code>	Integrate one or more generated process integrands
<code>gammaLoop generate</code>	Generate cross-section or amplitude integrands from a process specification
<code>gammaLoop generate xs</code>	Generate a cross-section integrand through the forward-scattering construction
<code>gammaLoop generate amp</code>	Generate an amplitude integrand for the supplied process specification
<code>gammaLoop generate existing</code>	Create another integrand from an existing process without regenerating its graphs
<code>gammaLoop select</code>	Select graph groups and write the selection to a new process or integrand

Command	Description
gammaLoop quit	End the interactive GammaLoop session without executing further commands
gammaLoop inspect	Inspect integrand contributions at one phase-space point or momentum configuration
gammaLoop approach	Sample an integrand while approaching a phase-space point along selected scaling axes
gammaLoop evaluate	Evaluate a vacuum amplitude analytically or numerically with Vakint
gammaLoop renormalize	Compute and export ultraviolet renormalization contributions for an amplitude
gammaLoop bench	Measure raw integrand evaluation throughput over random samples and selected worker counts
gammaLoop profile	Profile ultraviolet or infrared scaling limits of a generated integrand
gammaLoop profile ultra-violet	Sample ultraviolet scaling rays and report the fitted large-momentum behavior
gammaLoop profile bulk	Probe multiple infrared scaling limits and report their fitted behavior in one run
gammaLoop batch	Evaluate a process-definition file against an HPC batch input and write named results
gammaLoop !	Run an operating-system shell command from the interactive GammaLoop session

Settings

Path	Type	Default
cli.global.display_directive	string	"info"
cli.global.generation.compile.compilation_mode	string	"symjit"
cli.global.generation.compile.compiler	string	"g++"
cli.global.generation.compile.custom_compiler	array	[]
cli.global.generation.compile.fast_build	boolean	true
cli.global.generation.compile.optimization_level	string	"O2"
cli.global.generation.compile.unsafe_optimizations	boolean	true
cli.global.generation.evaluator.aboint_level	integer	0
cli.global.generation.evaluator.compile	boolean	false
cli.global.generation.evaluator.compile_integrations	integer or null	no serialized default
cli.global.generation.evaluator.disable_translation	boolean	true
cli.global.generation.evaluator.do_codegen	boolean	false
cli.global.generation.evaluator.do_gamma_loop_replacement	boolean	false
cli.global.generation.evaluator.horndir_generations	integer	1
cli.global.generation.evaluator.iterative_orientation	boolean	true
cli.global.generation.evaluator.max_integer_pair_cache	integer	1000000
cli.global.generation.evaluator.max_integer_pair_distance	integer	1000
cli.global.generation.evaluator.max_integer_scheme_variables	integer	500

Path	Type	Default
cli.global.generation.evaluator.n	integer	1
cli.global.generation.evaluator.serialization_mode	array	no serialized default
cli.global.generation.evaluator.stochastic	boolean	false
cli.global.generation.evaluator.submedian	boolean	false
cli.global.generation.evaluator.submedian_function_map	boolean	false
cli.global.generation.evaluator.testing_network_contract	string	"spiral_order_aware"
cli.global.generation.evaluator.verbose	boolean	false
cli.global.generation.feyngen.gamma_normalization	boolean	false
cli.global.generation.force_cuts	array	[]
cli.global.generation.orientation	pattern	starting
cli.global.generation.override_lmbooleans	boolean	false
cli.global.generation.threshold_submedian.assume_positive_external_energies	boolean	true
cli.global.generation.threshold_submedian.check_surface_at_generation	boolean	false
cli.global.generation.threshold_submedian.disable_integrated_ct	boolean	false
cli.global.generation.threshold_submedian.enable_thresholds	boolean	true
cli.global.generation.threshold_submedian.esurface_existence_threshold	integer	1
cli.global.generation.threshold_submedian.skip_thresholds_that_are_cuts	boolean	true
cli.global.generation.tropical_submedian.disable_tropical_generation	boolean	false
cli.global.generation.tropical_submedian.panic_on_fail	boolean	false
cli.global.generation.tropical_submedian.target_omega	boolean	1
cli.global.generation.uv.add_markers	boolean	false
cli.global.generation.uv.final_integrating	string	"ThreeD"
cli.global.generation.uv.generate_bondgraph	boolean	true
cli.global.generation.uv.inner_products	boolean	true
cli.global.generation.uv.keep_markers	boolean	true
cli.global.generation.uv.orchestrator	string	"hedge_poset"
cli.global.generation.uv.renormalization_prescription	string	"MUNDg_divergent"
cli.global.generation.uv.renormalization_prescription	string	"MUNDssive_power_divergent"
cli.global.generation.uv.renormalization_prescription	string	"MUNDssless_power_divergent"
cli.global.generation.uv.renormalization_prescription_overrides	array	[]
cli.global.generation.uv.softct	boolean	true
cli.global.generation.uv.subtract	boolean	true
cli.global.generation.uv.vakint.additional_normalization	boolean	true
cli.global.generation.uv.vakint.alphaloop.substitute_masters	boolean	true
cli.global.generation.uv.vakint.clean_trap_dir	boolean	true
cli.global.generation.uv.vakint.evaluation_methods	array	["alphaloop", "matad", "fmft"]
cli.global.generation.uv.vakint.expand_masters	boolean	true
cli.global.generation.uv.vakint.expand_substitute_masters	boolean	true
cli.global.generation.uv.vakint.forming_path	string	"form"
cli.global.generation.uv.vakint.matsubstitution	boolean	true

Path	Type	Default
cli.global.generation.uv.vakint.matboolexpand_master	boolean	true
cli.global.generation.uv.vakint.matbooleanstitute_hot	boolean	true
cli.global.generation.uv.vakint.matbooleanstitute_masters	boolean	true
cli.global.generation.uv.vakint.nostringization	string	"MSbar"
cli.global.generation.uv.vakint.pyintdeg_max_n_eval	integer	1000000000000
cli.global.generation.uv.vakint.pyintdeg_min_n_eval	integer	10000
cli.global.generation.uv.vakint.pybooleanquiet	boolean	true
cli.global.generation.uv.vakint.pynumber.relative_precision	number	7
cli.global.generation.uv.vakint.pystring.reuse_lexisting_output	string	reused default
cli.global.generation.uv.vakint.pystring.exe_path	string	"python3"
cli.global.generation.uv.vakint.pyintdeg_decimal_precision	integer	10
cli.global.generation.uv.vakint.temporary_directory	string	no serialized default
cli.global.generation.vector_polarization_sum_gauge	string	"LightLikeAxial"
cli.global.log_style.full_line_source	boolean	false
cli.global.log_style.include_fields	boolean	false
cli.global.log_style.log_format	string	"Long"
cli.global.log_style.short_timestamp	boolean	false
cli.global.logfile_directive	string	"off"
cli.global.n_cores.compile	integer	1
cli.global.n_cores.feyngen	integer	1
cli.global.n_cores.generate	integer	1
cli.global.n_cores.integrate	integer	1
cli.override_state	boolean	false
cli.state.folder	string	"./gammaLoop_state"
cli.state.name	string null	""
cli.try_strings	boolean	true
runtime.general.additional_param_values	array	[]
runtime.general.debug_cache	boolean	false
runtime.general.disable_flux_factor	boolean	false
runtime.general.enable_cache	boolean	false
runtime.general.evaluator_method	string	"SingleParametric"
runtime.general.generate_events	boolean	false
runtime.general.integral_unit	string	"auto"
runtime.general.load_compiled_cff	boolean	false
runtime.general.m_uv	number	1000.0
runtime.general.mu_r	number	1000.0
runtime.general.orientation_path	null string	no serialized default
runtime.general.renormalization_localization_scale	number	1000.0
runtime.general.store_additional_weights_in_event	boolean	false
runtime.general.use_ltd	boolean	false

Path	Type	Default
runtime.h_function.enabled_damping	boolean	true
runtime.h_function.function	string	"poly_exponential"
runtime.h_function.power	integer null	no serialized default
runtime.h_function.sigma	number	1.0
runtime.integrand.n_3d_momenta	integer	no serialized default
runtime.integrator.bin_number_evolutionary	integer null	no serialized default
runtime.integrator.continuous_dim_learning_rate	number	1.0
runtime.integrator.discrete_dim_learning_rate	number	1.0
runtime.integrator.integrated_phases	string	"real"
runtime.integrator.max_prob_ratio	number	30.0
runtime.integrator.min_samples_for_integration	integer	1000
runtime.integrator.n_bins	integer	64
runtime.integrator.n_increase	integer	10000
runtime.integrator.n_max	integer	10000000000
runtime.integrator.n_start	integer	100000
runtime.integrator.observables_output_format	string	["json"]
runtime.integrator.seed	integer	69
runtime.integrator.show_max_wgt_info	boolean	true
runtime.integrator.target_absolute_number	number	no serialized default
runtime.integrator.target_relative_number	number	no serialized default
runtime.integrator.train_on_avg	boolean	false
runtime.kinematics.e_cm	number	64.0
runtime.kinematics.externals.data_activities	array	[]
runtime.kinematics.externals.data_input_member_settings	object	{large_deformation_check: 10}
runtime.kinematics.externals.data_improvement_settings	object	{mode: "none"}
runtime.kinematics.externals.data_improvement_settings	object	{only_warn_on_large_deformation: false}
runtime.kinematics.externals.data_momenta	array	[]
runtime.kinematics.externals.type	string	"constant"
runtime.model	object	{}
runtime.observables	object	{}
runtime.quantities	object	{}
runtime.sampling.alpha	number	3.0
runtime.sampling.b	number	1.0
runtime.sampling.coordinate_system	string	"spherical"
runtime.sampling.graph_names	array	[]
runtime.sampling.graphs	string	"summed"
runtime.sampling.lmb_basis_ids	object	no serialized default
runtime.sampling.lmb_channel_weights	string	"ose"
runtime.sampling.lmb_channels	string	"summed"
runtime.sampling.lmb_multichanneling	boolean	false

Path	Type	Default
runtime.sampling.mapping	string	"linear"
runtime.sampling.orientations	string	"summed"
runtime.sampling.power	number	1.0
runtime.selectors	object	{}
runtime.stability.check_on_norm	boolean	true
runtime.stability.escalate_if_exact_zero	boolean	false
runtime.stability.levels	array	[{"escalate_for_large_weight_threshold": 1e-6}, {"escalate_for_large_weight_threshold": 1e-7}, {"escalate_for_large_weight_threshold": 1e-8}]
runtime.stability.loop_momenta_normalisation_factor	number	0.1
runtime.stability.recording.record_lowest_stability_level	boolean	no serialized default
runtime.stability.recording.record_lowest_momenta_escalation	boolean	no serialized default
runtime.stability.recording.record_failed_results	boolean	no serialized default
runtime.stability.rotation_axis	array	[{"alpha": 0.1, "beta": 0.2, "gamma": 0.3}]
runtime.subtraction.disable_threshold_subtraction	boolean	false
runtime.subtraction.esurface_existence_threshold	number	1e-7
runtime.subtraction.integrated_ct_settings.range.h	function	function settings.enabled_dampening
runtime.subtraction.integrated_ct_settings.range.h	function	function exponential
runtime.subtraction.integrated_ct_settings.range.h	function	function serialised settings power
runtime.subtraction.integrated_ct_settings.range.h	function	function settings.sigma
runtime.subtraction.integrated_ct_settings.range.type	string	"infinite"
runtime.subtraction.local_ct_settings.localisation.dynamic_width	boolean	false
runtime.subtraction.local_ct_settings.localisation.force_uv_dampers_to_one	boolean	false
runtime.subtraction.local_ct_settings.localisation.gaussian_width	number	10.0
runtime.subtraction.local_ct_settings.localisation.oliver_width	number	10.0
runtime.subtraction.overlap_settings.back_global_center	boolean	true
runtime.subtraction.overlap_settings.forward_global_center	boolean	no serialized default
runtime.subtraction.overlap_settings.only_origin	boolean	true
runtime.subtraction.overlap_settings.only_origin_always	boolean	false
runtime.subtraction.radial_root_residual_tolerance	number	64.0

Version information

Save these identifiers with published results and include them in issue reports so that collaborators can reproduce the same software environment.

- **Documentation channel:** latest
- **Documentation route:** /products/gammaploop/latest/
- **Source revision:** 28275fc5d5154faa43a19dba0233bacd0dafbf1f

Package versions and enabled features

- gammaploop/gammaplooprs — 0.3.4 (no optional features)
- gammaploop/gammaploop-api — 0.1.0 (features: cli)
- gammaploop/gammaploop — 0.3.4 (features: python_api, python_abi, python_stubgen)
- linnet/linnet — 0.17.0 (features: nodestore-vec, serde, bincode, drawing, symbolica)
- linnet/linnet-py — 0.1.0 (features: extension-module, abi3-py310)
- spenso/spenso — 0.6.0 (features: shadowing)
- spenso/spenso-macros — 0.3.2 (features: shadowing)
- spenso/spenso-hep-lib — 0.1.7 (no optional features)
- spenso/spynso3 — 0.1.2 (features: python_stubgen)
- idenso/idenso — 0.3.0 (features: bincode, reference-cases)
- idenso/idenso — 0.3.0 (features: python, python_stubgen)
- vakint/vakint — 0.1.2 (no optional features)
- vakint/vakint — 0.1.2 (features: symbolica_community_module, python_stubgen)